

Programmatic Generation of GPU Matrix Multiplication Primitives

by

Rishab Parthasarathy

S.B. Electrical Engineering and Computer Science and Physics,
Massachusetts Institute of Technology, 2026

Submitted to the Department of Electrical Engineering and Computer Science
in partial fulfillment of the requirements for the degree of

Master of Engineering in Electrical Engineering and Computer Science

at the

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

September 2026

© 2026 Rishab Parthasarathy. All rights reserved.

The author hereby grants to MIT a nonexclusive, worldwide, irrevocable, royalty-free license to exercise any and all rights under copyright, including to reproduce, preserve, distribute and publicly display copies of the thesis, or release the thesis under an open-access license.

Authored by: Rishab Parthasarathy
Department of Electrical Engineering and Computer Science
August 11, 2026

Certified by: Saman Amarasinghe
Professor, Department of Electrical Engineering and Computer Science
Thesis Supervisor

Accepted by: Katrina LaCurts
Chair, Master of Engineering Thesis Committee

Programmatic Generation of GPU Matrix Multiplication Primitives

by

Rishab Parthasarathy

Submitted to the Department of Electrical Engineering and Computer Science
on August 11, 2026 in partial fulfillment of the requirements for the degree of
Master of Engineering in Electrical Engineering and Computer Science

ABSTRACT

High-performance GPU kernels in CUDA, especially on Hopper and later GPUs, rely on a programming model that is meaningfully different from earlier CPU and GPU code. In particular, strong performance depends on four concepts: asynchronous bulk data movement with the Tensor Memory Accelerator (TMA), barrier synchronization between producer and consumer warpgroups, warpgroup MMA (WGMMMA) instructions that operate across 128 threads, and explicit register partitioning between producer and consumer roles. As a result, many modern domain-specific language (DSL) generators require the programmer to think in terms of matrix tiling and underlying hardware constraints, leading to programming models that are often unintuitive.

This thesis instead explores performant GPU code generation for Hopper GPUs directly from loopnests in BuildIt, a type-based multi-stage C++ framework for constructing embedded DSLs. Specifically, the work proposed in this thesis allows BuildIt to convert simple loopnest code to pipelined GPU kernels, reaching 82 to 96 percent of state-of-the-art kernel performance for square problems with sizes between 2048 and 8192 and reducing user code written by up to 20x. Using custom compiler annotations, this thesis also supports a variety of input data types, memory layouts, and matrix dimensions. As a whole, this thesis thus demonstrates that many low-level kernel attributes can be directly extracted from loopnest parameters, supporting performant kernels with a simple programming model.

Thesis Supervisor: Saman Amarasinghe

Title: Professor, Department of Electrical Engineering and Computer Science

Acknowledgments

I would like to take this space to thank my advisor, Professor Saman Amarasinghe, and mentor, Ajay Brahmakshatriya, for giving me the opportunity to write this thesis and learn more about compilers in general. Thank you both for taking the time to always sit down with me and work through problems step-by-step. Your feedback and mentorship has been invaluable for this thesis and my development as a researcher.

Beyond this thesis, I would like to thank Professors Daniel Sanchez and Jacob Andreas for having given me the opportunity to pursue research at MIT and explore my varied interests, even when I wasn't sure what I actually wanted to pursue.

I would also especially like to thank Professor Jonathan Ragan-Kelley for introducing me to the world of accelerated computing and GPU performance engineering, from your class in junior year to TAing the same class my senior year. Without 6.S894, I would have never pursued this direction for my thesis.

Finally, above all, I would like to thank the friends I made at MIT and my family for sticking by me and supporting me over the past few years. Thank you for making MIT the best time of my life.

Contents

<i>List of Figures</i>	9
<i>List of Tables</i>	11
1 Introduction	13
1.1 BuildIt and Staged GPU DSLs	15
1.2 CuTe and the Hopper Programming Model	15
1.3 Contributions	16
1.4 Thesis Organization	16
2 Background	19
2.1 Multi-Stage Programming in BuildIt	19
2.1.1 CUDA Extraction	19
2.2 The Hopper Programming Model	20
2.2.1 Overall GPU Architecture	21
2.2.2 The Tensor Memory Accelerator	21
2.2.3 Warpgroup MMA	22
2.2.4 Asynchronous Barriers and Warp Specialization	22
2.3 CuTe Primitives	23
3 Related Work	25
3.1 DSL Generators and Scheduling Algorithms	25
3.1.1 Scheduling Algorithms	25
3.2 GPU Programming Models	26
3.2.1 Performance Oriented GPU DSLs	26
3.2.2 Tensor Compilers and Languages	27
3.3 Hand-Optimized Kernels	28
4 System Design and Implementation	29
4.1 Overview and Design Philosophy	29
4.2 GPU Memory Buffers	30
4.3 Staged CuTe Helper Functions	31
4.4 The <code>extract_cute</code> Loopnest Analysis	33
4.4.1 The Annotated Input Pattern	34
4.4.2 Extracting Loop Bounds	35

4.4.3	Enforcing Loop Invariants	36
4.4.4	Inferring Operand Types	37
4.4.5	Inferring Operand Layouts	38
4.5	The <code>extract_cute</code> Output Pass	39
4.5.1	Emitting the Kernel	39
4.5.2	CUDA Extraction Changes	41
4.6	Warp Specialization	41
5	Evaluation	45
5.1	Experimental Setup	45
5.2	Correctness Validation	45
5.3	Programmability	47
5.4	Performance	48
5.4.1	Baseline Kernel	48
5.4.2	Better CTA Tiling	50
5.4.3	Warp Specialization and Tile Size Changes	51
5.4.4	Storing Matrices in the Right Layout	51
5.4.5	End-to-End Performance	52
5.5	Final Thoughts	54
6	Limitations and Future Work	55
6.1	Generalizing the Pass	55
6.1.1	Advanced Hardware Features	55
6.1.2	Divisibility Constraints	55
6.2	Fused Kernels	56
6.3	DSL Integration	56
7	Conclusion	59
	<i>Bibliography</i>	61

List of Figures

Figure 1	CUDA Kernel Complexity	14
Figure 2	Staging Optimizations in BuildIt	19
Figure 3	The <code>CUDA_KERNEL</code> annotation pattern in BuildIt.	20
Figure 4	A transformed CUDA kernel from BuildIt	20
Figure 5	Simplified H100 architecture	21
Figure 6	Compilation pipeline with the new extract CuTe pass	30
Figure 7	Example of BuildIt CUDA memory annotations.	31
Figure 8	An example <code>memcpy</code> kernel using shared memory	31
Figure 9	The templated CuTe helper structure used by BuildIt	32
Figure 10	The generic single-warpgroup <code>gemm_tile</code> implementation	33
Figure 11	A loopnest GEMM implementation, illustrating use of <code>WGMMMA_INNER</code> and <code>WGMMMA_OUTER</code>	34
Figure 12	Example of loop bound extraction for a complex loopnest	36
Figure 13	Simple type markings in BuildIt	37
Figure 14	A mixed precision BF16 type in BuildIt	37
Figure 15	Generated CuTe code for a baseline matmul	40
Figure 16	Warp-specialized BuildIt input loopnest	42
Figure 17	The warp-specialized <code>gemm_tile_ws</code> implementation	43
Figure 18	Baseline kernel used for performance benchmarking	49
Figure 19	SASS implications of the <code>extract_cute</code> annotations	50
Figure 20	A simple 2D tiling of the output matrix over CTAs	50
Figure 21	SASS changes in the warp-specialized kernel	51
Figure 22	K-major vs. MN-major B matrix in SASS	52
Figure 23	Adjacency constraints on row vs. column major C-matrix in SASS	52
Figure 24	The final kernel reaching near-cuBLAS performance	54
Figure 25	The proposed einsum-to-CuTe pipeline	57

List of Tables

Table 1	Affine analysis results from the expression tree	35
Table 2	Correctness validation for different compute configurations, memory layouts, and problem sizes	46
Table 3	Lines of code comparison across different GEMM programming methods . .	47
Table 4	Code changes across different GEMM configurations	48
Table 5	Performance results for different BuildIt kernel configurations	53

Chapter 1

Introduction

On recent generations of GPUs, high-performance programming requires a programmer to not only write multithreaded code, but also manage the synchronization of hardware co-processors such as the Tensor Memory Accelerator (TMA) and Warpgroup Matrix-Multiply-Accumulate (WGMMA) [11]. These hardware co-processors perform specific operations fine-tuned for accelerating matrix multiplication, which is the main computational bottleneck of many current AI workloads.

Specifically, the TMA reduces the number of threads required to move large multidimensional tiles between global and shared memory, while WGMMA allows a warpgroup of 128 threads to collectively perform large tensor-core matrix operations. These features are designed to work in lockstep, as some parts of a kernel, termed *producers*, can read data from memory using the TMA, while other parts of a kernel, termed *consumers*, can simultaneously use the WGMMA to perform computations on the aforementioned data.

While this specialization in hardware has resulted in significant improvements in GPU performance, it has also resulted in increasingly complex code. Take the example of a general matrix multiplication (GEMM). Intuitively, a GEMM should be easily representable as a simple loopnest over the M, N, K dimensions. However, a performant implementation on an NVIDIA Hopper GPU must choose tile shapes for the TMA and WGMMA, stage data in shared memory, coordinate asynchronous copies, issue tensor-core operations, and overlap memory movement with computation. Each of these choices in layout or synchronization can change performance by orders of magnitude, and even impact correctness. This difference is shown in Figure 1, where Figure 1b is the skeleton of a kernel that totals around 400 lines of code, just to perform the same simple loopnest computation displayed in Figure 1a.

Figure 1a: A GEMM loopnest.

```
1 for (int i = 0; i < M; i++)
2   for (int j = 0; j < N; j++)
3     for (int k = 0; k < K; k++)
4       C[i*N+j] += A[i*K+k] * B[k*N + j];
```

Figure 1b: A skeleton of a hand-written GEMM kernel [12, 21].

```
1 __global__ void __launch_bounds__(total_threads)
2 gemm(const __grid_constant__ CUtensorMap tma_a,
3       const __grid_constant__ CUtensorMap tma_b, bf16 *C, int M, int N, int K) {
4   extern __shared__ char smem[];
5
6   // 1. create swizzled smem tiles for A/B matrices
7   // 2. initialize barriers + add arrival conditions
8
9   // producer warpgroup
10  if (warpgroup() == 0) {
11    setmaxnreg_dealloc<n_producer_regs>();
12    for (kt = 0; kt < K/TILE_K; kt++) {
13
14      // 1. wait for the previous pipeline stage to complete
15      // 2. reinitialize pipeline barriers for the TMA
16      // 3. issue correct TMA loads for A/B matrices
17
18    }
19  }
20
21  // consumer warpgroups
22  else {
23    setmaxnreg_alloc<n_consumer_regs>();
24    float acc[N_REG] = {0};
25    for (kt = 0; kt < K/TILE_K; kt++) {
26
27      // 1. wait for the TMA loads to complete
28      // 2. create shared memory descriptors
29      // 3. issue WGMMAs operations
30      // 4. wait for the WGMMAs to commit and send the pipeline back to producer
31
32    }
33
34    // Epilogue: map every single accumulated value back to global C
35
36  }
37 }
38
39 // Host code:
40 // 1. Create TMA descriptors, allocate dynamic shared memory
41 // 2. Launch kernel with appropriate memory buffers, etc.
```

Figure 1: (a) The simple loopnest implementation of a GEMM. (b) The structure of the equivalent hand-written Hopper kernel, where each comment represents significant amounts of hand-rolled low-level API and synchronization code, totalling at around 400 lines for the full kernel [12, 21].

As a whole, Figure 1 illustrates the tension between abstraction and complexity in modern GPU programming. While targeted management of low-level APIs is typically needed for

state-of-the-art performance, these APIs are simultaneously unintuitive and difficult to use, significantly increasing the load on the programmer.

In this thesis, we thus explore the key tradeoff between abstraction and complexity by studying how to generate performant GPU code from simple baseline loopnests, adding this functionality to the BuildIt framework for generating domain-specific languages (DSLs).

1.1 BuildIt and Staged GPU DSLs

BuildIt is a multi-stage generator for domain-specific languages (DSLs) in C++ [1]. In general, a BuildIt program uses dynamic variables, annotated using `dyn_var<T>`, to construct an abstract syntax tree based on inferred program execution, which can be used to lower the program into C, C++, or CUDA using a set of compiler optimization passes. As of today, BuildIt’s existing CUDA support follows an annotation-driven model: when a user marks a loop with `CUDA_KERNEL`, BuildIt extracts a device kernel, substitutes CUDA indices, emits a launch, and keeps the source program close to ordinary C++. In this thesis, we ask whether this same annotation-driven model can be used to generate code for Hopper-generation GPUs.

1.2 CuTe and the Hopper Programming Model

To produce performant GPU code, bare-metal CUDA exposes not only high-level multithreading abstractions, but also low-level APIs for thread blocks, synchronization primitives, and the memory hierarchy [13]. These unique APIs are especially prominent on Hopper and later GPUs, where TMA, WGMMMA, and asynchronous barrier operations are accessed through PTX, an intermediate representation that sits close to NVIDIA’s proprietary SASS assembly language [14]. As a result, to generate many features required for performant Hopper kernels, a kernel generator must output inline PTX, which while structured, is extremely complex. For example, to properly output inline PTX for the TMA and WGMMMA, the compiler must know which buffers are in shared memory, what memory layout is used for the matrices, and the pipelining algorithm used in the code.

Because of these hardware constraints, outputting inline PTX is not a natural target for high-level language generators like BuildIt. Hence, this thesis instead targets CuTe, which exposes GPU programming primitives through basic C++ building blocks. CuTe exposes primitives for tensor layouts, tiled copies, tiled matrix multiply, and other critical operations, and is used throughout CUTLASS, NVIDIA’s state-of-the-art high-performance matrix multiply library [15]. Thus, targeting CuTe enables this thesis to focus on staged extraction of loopnest GPU kernels and generation of kernel workflows, while CuTe supplements BuildIt by providing robust instantiations of PTX operations.

1.3 Contributions

Overall, this thesis makes three main contributions:

1. **GPU memory support in BuildIt’s intermediate representation.** We add a lightweight annotation framework on variables that carries qualifiers such as `__shared__` and `const __grid_constant__` through every existing compiler pass unchanged (Section 4.2).
2. **A staged helper library that lowers to CuTe.** Rather than requiring BuildIt to reason about complex CuTe code, we design a templated data structure, `CuteHopperGemm`, that provides all relevant matrix multiplication functionality for BuildIt (Sections 4.3 and 4.6).
3. **A custom analysis pass that converts loopnests to CuTe.** Finally, we implement `extract_cute`, an analysis pass that recognizes an annotated matmul loopnest and rewrites it into calls to the helper library. Using an affine analysis of the loop bounds, the pass recovers the target memory layout and data type for all operands, generalizing to row and column-major layouts for each operand, BF16 outputs with FP32 accumulation, and batched multiplication through three-dimensional TMA descriptors. Depending on the number of available threads, the pass can also warp-specialize the kernels it generates (Sections 4.4 through 4.6).

1.4 Thesis Organization

The remainder of this thesis is organized as follows:

Chapter 2 (Background) provides an overview of BuildIt, Hopper GPU programming, and the CuTe infrastructure used in this thesis.

Chapter 3 (Related Work) discusses current advancements across DSL generators, GPU performance engineering, and tensor compilers, comparing and contrasting the tradeoffs made in these works versus this thesis.

Chapter 4 (System Design and Implementation) walks step-by-step through the design decisions made when implementing the BuildIt CuTe pass, from affine loopnest analysis to warp specialization.

Chapter 5 (Evaluation) provides a comparison of this thesis to current state-of-the-art cuBLAS kernels across correctness, programmability, and performance. In terms of performance, this chapter follows a sequence of kernel optimizations made using the simple loopnest programming model, culminating at near state-of-the-art performance.

Chapter 6 (Limitations and Future Work) discusses potential avenues for extending the work in this thesis by using advanced GPU capabilities, pursuing kernel fusion, or writing a higher-level DSL.

Chapter 7 (Conclusion) provides our final thoughts and summarizes the lessons learned from implementing the BuildIt CuTe pass.

Chapter 2

Background

In this chapter, we provide a starting point for the algorithmic discussion in Chapter 4 by building on the details from Chapter 1.

2.1 Multi-Stage Programming in BuildIt

As discussed previously, BuildIt [1] is a C++ library for multi-stage programming, especially for DSL generation.

BuildIt’s key innovation is that it divides values into two kinds, `static_var` and `dyn_var`. The key invariant is that a `static_var` holds a value that is guaranteed to be known during the first stage of compilation, so all computations over it can be evaluated away. In contrast, a `dyn_var` possesses no such guarantees, so BuildIt instead uses the present `dyn_vars` to build an abstract syntax tree (AST) of BuildIt IR nodes. This delineation between static and dynamic variables is particularly effective because it allows DSLs built on BuildIt to construct fast, specialized code when executed. For example, loops and branches over static variables can be completely unrolled, resulting in code like in Figure 2.

```
1 // Staged function
2 dyn_var<int> power(dyn_var<int> x,
3                   static_var<int> n) {
4     dyn_var<int> result = 1;
5     for (static_var<int> i = 0;
6         i < n; i = i + 1)
7         result = result * x;
8     return result;
9 }
10
// Generated code for n = 5
int power_5(int arg0) {
    int var1 = 1;
    var1 = var1 * arg0;
    var1 = var1 * arg0;
    var1 = var1 * arg0;
    var1 = var1 * arg0;
    var1 = var1 * arg0;
    return var1;
}
```

Figure 2: Staging in BuildIt allows the the `static_var` loop to be completely analyzed and unrolled in the first phase, while only operations on `dyn_var` survive in the generated program.

2.1.1 CUDA Extraction

Currently, BuildIt’s CUDA extraction pass requires the user to write the computation as a `CUDA_KERNEL` annotated nest of two ordinary loops, with an outer loop over thread blocks or cooperative thread arrays (CTAs) and an inner loop over threads, as shown in Figure 3.

```

1  builder::annotate(CUDA_KERNEL);
2  for (dyn_var<int> cta = 0; cta < num_blocks; cta = cta + 1) {
3      for (dyn_var<int> tid = 0; tid < 512; tid = tid + 1) {
4          // ... body, written as if sequential ..., uses buf0 and buf1
5      }
6  }

```

Figure 3: The outer loop describes the number of CTAs, the inner loop describes the number of threads, and the body is written as sequential code describing the behavior of an individual thread.

Within BuildIt itself, the `extract_cuda_from` pass identifies all pairs of loops annotated with `CUDA_KERNEL` and moves the body into a new CUDA device-side (GPU-side) function. Since the `cta` and `tid` variables indicate the CTA and thread numbers on the GPU, the pass replaces reads of those two variables with `blockIdx.x` and `threadIdx.x`, the equivalent values in the CUDA programming API. The pass also computes the set of variables that the body references but does not declare, which become kernel parameters, as they must be passed from the CPU host to the GPU. For example, for the code depicted in Figure 3 transfers to Figure 4.

```

1  void __global__ cuda_kernel_0 (int* arg0, int* arg1) {
2      // body, with cta -> blockIdx.x and tid -> threadIdx.x
3  }
4  ...
5  cuda_kernel_0<<<num_blocks, 512>>>(buf0, buf1); // replaces the two loops

```

Figure 4: From Figure 3, the `cta` and `tid` parameters have become constraints of the CUDA kernel launch, and the `buf0/buf1` used within the kernel have become arguments to the CUDA kernel.

In this thesis, we extend the framework presented by this model, since it provides an already working mechanism for kernel outlining, argument wiring, and launch emission. Overall, we fill the gaps that this model does not cover, such as emitting shared memory descriptors, organizing operations performed collectively by a warpgroup, and signaling asynchronous work through barriers. These augmentations of the BuildIt CUDA extraction model are covered in more depth in Chapter 4.

2.2 The Hopper Programming Model

This section describes the additions to the programming model created by the Hopper generation of NVIDIA GPUs in more depth compared to Chapter 1. We focus on the H100 GPU, which we provide a summary of in Figure 5.

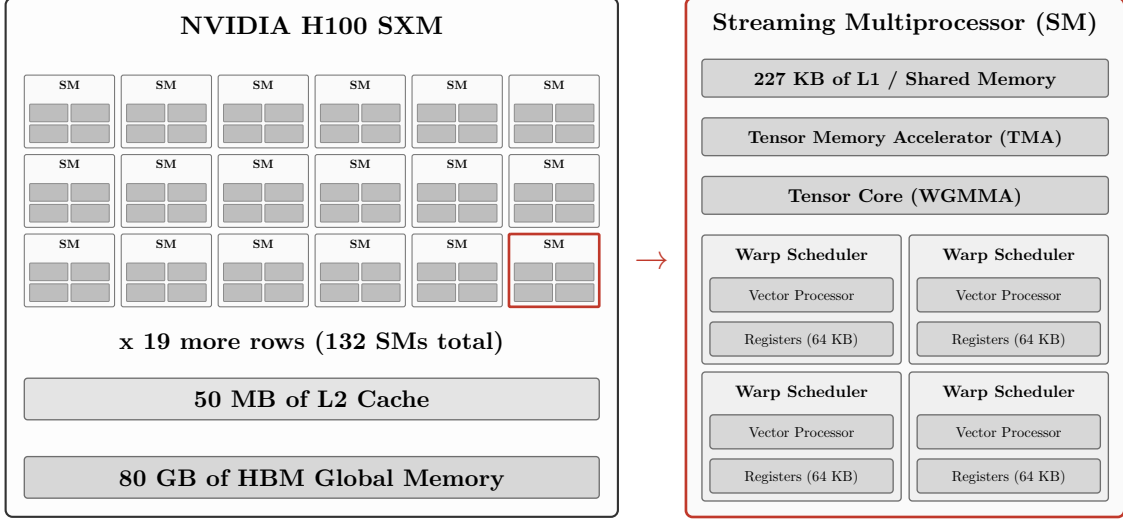


Figure 5: A simplified view of the H100 architecture, with 132 streaming multiprocessors (SMs), a 50 MB L2 cache, and 80 GB of global memory. Each SM contains 227 KB of L1 cache or shared memory, a tensor memory accelerator (TMA), a tensor core (represented across the whole SM because we use warpgroup matrix multiplication (WGMMMA)), and four warp schedulers with 64 KB of registers and a vector processor.

2.2.1 Overall GPU Architecture

Each NVIDIA GPU, including the H100, is divided into multiple streaming multiprocessors (SMs), which are the GPU equivalent of a CPU core. Each SM consists of four vector processors split across four warp schedulers, which execute threads using a Single Instruction, Multiple Thread (SIMT) compute model. The whole SM is backed by a non-coherent L1 cache, which is frequently configured into a state known as shared memory. When configured as shared memory, the L1 cache per SM can be used as a scratchpad for the programmer to synchronize between different warps (a unit of 32 threads executing in lockstep) executing on different warp schedulers. Specifically, on the H100 SXM GPU, there are 132 SMs, each of which has 64K 32-bit registers (or 256 KB of register file) in total and up to 227 KB of usable shared memory per thread block. Outside of the base vector processor, each of the SMs also contain both a TMA and WGMMMA unit, which are explained in more depth in the coming sections [11].

2.2.2 The Tensor Memory Accelerator

As mentioned above, each SM contains a tensor memory accelerator, or TMA, which is a memory coprocessor that asynchronously transfers multidimensional tiles of data between global and shared memory. Unlike previous generations of GPUs, where address calculation and coordinating memory transfers became a key bottleneck on the compute capabilities of the GPU, the TMA coprocessor allows Hopper and later generations of GPUs to fully overlap and pipeline memory transfers with large mathematical computations. Specifically, instead of requiring all threads per warp to participate in memory transfers, a single thread is allowed to issue a TMA copy [14]. Instead of requiring the programmer to manually

coalesce and organize memory accesses, the TMA allows the programmer to package all information about the memory layout in a host-side `CUTensorMap` descriptor, which captures a tensor’s base address, shape, strides, and more. Especially important is that the TMA can autonomously handle *swizzling*, which minimizes bank conflicts in shared memory by permuting the memory accesses between threads.

In terms of synchronization, the TMA uses a concept known as *split barriers*. Whereas typical barriers require all threads to arrive at a synchronization point, split barriers allow a barrier to wait on a specific number of threads to arrive or a specific amount of data to be transferred, which works naturally with the TMA’s variable sized memory transfers. In the Hopper architecture, split barriers (also known as `mbarriers`), are functionally counters with a phase bit, which flips when the barrier wait is complete. This phase bit enables straightforward pipelining of operations, as the user knows which phase to expect from earlier pipeline stages to prevent reading stale or incorrect data.

2.2.3 Warpgroup MMA

Beyond the TMA, each SM also contains a matrix multiplication unit (WGMMA), which is shared across 128 threads executing across four warps (known as a warpgroup). Like the TMA, the WGMMA executes fully asynchronously, where the matrix operands are read directly from shared memory through *matrix descriptors* that are very similar to the tensor maps used by the TMA. However, one key difference is that the WGMMA can only execute on a few fixed sizes of matrix multiplication, and in this thesis, we focus on problems of the size $64 \times N \times 16$ for N up to 256. In terms of synchronization, the WGMMA unit relies on the concept of commit groups, where certain WGMMA operations can be batched and committed together in a group, and subsequent operations can wait for different commit groups to complete.

One key constraint of the WGMMA is that because it operates on very large input/output matrices, where the output matrix is stored in registers scattered across all 128 threads, it inherits significant register pressure constraints. For example, a 128×256 FP32 accumulator tile occupies 128 registers in every thread of the warpgroup. These registers are also supplemented by registers for storing addressing information and pipeline state. This means that even a uniform allocation of the 64K registers across 3 warpgroups (384 threads) would suffer from significant register pressure, as $\frac{65536}{384} \approx 170$ is very close to the combined register count of the output tile and other state. As a result, programmers adopt a philosophy known as warp specialization.

2.2.4 Asynchronous Barriers and Warp Specialization

While warp specialization is not unique to Hopper generation GPUs, it is especially important on these GPUs because of the register pressure created by the WGMMA. In a warp-specialized kernel, instead of having each warp execute the same instructions (and

thus take the same number of registers), each warp or warpgroup can execute a different set of operations with different memory and register constraints.

For example, in a warp-specialized matrix multiplication kernel with multiple warpgroups, we might assign warpgroup 0 to be a producer. In this case, warpgroup 0 would issue TMA copies and create a pipeline of shared-memory stages, never touching the tensor cores and thus using very few registers. The remaining warpgroups can be used as consumers, where they wait on the TMA stage to arrive in the pipeline and then issue corresponding WGMMMA operations, which lets the producer reuse the stage. Hence, in warp specialized code, the programmer can use PTX instructions like `setmaxnreg` to partition registers unequally between warpgroups at runtime, enabling the consumers to use more registers than otherwise possible and thus use larger matrix multiplication sizes [14].

2.3 CuTe Primitives

While all of the Hopper programming model tools above are crucial for writing performant kernels, the majority of these functionalities are only supported in CUDA C++ through inline PTX, which is difficult to write and hard to debug. To address this, NVIDIA provides CuTe, as described in the introduction, which wraps inline PTX functionality through tensor layouts in ordinary C++ [15].

To write CuTe, the user specifies tensor layouts, which are essentially memory patterns described using a hierarchy of shapes and strides. For example, a CuTe tensor layout that describes a 64×64 tile stored in row-major format could be described using `Layout<Shape<_64, _64>, Stride<_64, _1>>`, where the `<_64, _1>` indicates row-majority because two adjacent elements in the first dimension are 64 elements apart while two adjacent elements in the second dimension are actually adjacent in memory. Hence, a column-major matrix could instead be easily represented by just switching the stride to `Stride<_1, _64>`. The key advantage of these CuTe tensor layouts is that because they are composable and deterministic, many layout algebra-based tensor operations, such as tiling, applying swizzling, and partitioning the tile over 128 threads can be evaluated entirely at compile time, enabling easy checking of addressing and correctness. In this thesis, we primarily utilize three functionalities from CuTe for utilizing the unique features on Hopper generation GPUs:

1. `make_tma_copy`: Constructs a TMA descriptor from global memory and shared memory tensor layouts, abstracting away the `CUTensorMap`.
2. `cute::GMMMA::ss_op_selector`: Uses the memory layouts and data types of the different operands to select the correct WGMMMA operation available on the H100.
3. `cute::gemm` performs the entire WGMMMA operation for a tile of work, strictly based on the MMA object created using the memory layouts.

Since these functions rely on variables and layout templates that are available at compile time, we can leverage CuTe natively by building generic template machinery, which is discussed in more depth in Chapter 4, making CuTe a logical fit for this use case.

Chapter 3

Related Work

3.1 DSL Generators and Scheduling Algorithms

Previous work in DSL generation has demonstrated the ability of C++ extensions in specifying a wide variety of generic DSLs. One recent work in this direction is BuildIt, which is the foundation of this thesis [1]. Specifically, BuildIt programs are compiled in two stages. In the first stage, BuildIt uses static variable values and dynamic variable annotations to perform a variety of compiler passes that construct a specific, optimized abstract syntax tree (AST) for a given program. In the second stage, BuildIt uses a standard compiler to convert the lowered code from the first stage into machine code.

Apart from BuildIt, Delite is an earlier example of generating performance-oriented embedded DSLs. Compared to BuildIt, Delite provides a larger set of compiler primitives, including parallel patterns, code generators, and heterogeneous execution [25]. However, Delite is less embedded into standard C++, and is heavier than BuildIt, which illustrates a key tradeoff in DSL generators between ease of use and performance. In this thesis, even working with BuildIt, we address the same issue, generating high-performance CUDA kernels with a BuildIt extension that remains simple to use.

3.1.1 Scheduling Algorithms

In order to improve the performance of generalized DSLs, Halide popularized the idea of separating algorithmic details from software and hardware scheduling, demonstrating the effectiveness of this idea on image processing pipelines [19]. Building on Halide, TVM implemented the same technique for deep-learning compilers, creating auto-tuned schedules for large machine learning models [2].

More recent work on scheduling algorithms has since gone in two divergent directions. One direction, pursued by Exo and Fireiron, establishes dataflow constraints like memory movement and vectorization as generic scheduling primitives, exposing full-low level control of scheduling to the programmer. These methods give programmers the ability to write their own fine-grained hardware schedules, but simultaneously require significant complexity in the code being written [8, 10].

The other direction pursues full autogeneration of schedules, with some approaches like Ansor or Bolt using a space of pre-generated schedules (be it from CUTLASS or TVM), to search for the optimal configuration of a given target kernel [2, 15, 31, 33]. Other approaches, like ROLLER and Mirage, instead optimize over base hardware parameters, deriving optimal kernel dimensions from first principles [30, 35]. As a whole, to autogenerate a schedule, all of these approaches typically operate on a full computation graph of the machine learning workload being computed, which in practice often requires using a heavyweight library like PyTorch or HuggingFace.

In this thesis, we take a middle ground between these two perspectives. We give the user some low-level programmability and make the interface lightweight through BuildIt, not requiring a large secondary library to be connected. However, we also make the design choice to have the compiler fully own hardware scheduling outside of loopnest dimensions and tiling, significantly simplifying the code the user must write.

3.2 GPU Programming Models

Beyond scheduling and automated machine learning compilers, recent work in GPU programming models has focused both on performant low-level GPU programming interfaces and also high-level tensor compilers. In this section, we lay out these two directions and the lessons we learn from both.

3.2.1 Performance Oriented GPU DSLs

Many recent works have focused on creating specialized DSLs for performant GPU programming. One direction of work has focused on changing the programming model from CUDA threads to a tiled interface more similar to the memory layout used by standard GPU kernels. For example, NVIDIA’s recent work on cuTile is a Python-based DSL and tile programming model that enables the programmer to invoke tensor cores and the TMA without explicitly managing tensor memory descriptors, which are inferred from the tiled memory layout provided by the user [16]. Another recent work from NVIDIA is CuTe DSL, which provides a similar Python DSL for generating CUDA, which instead just-in-time (JIT) compiles a Python version of the CuTe API [17]. Both of these DSLs focus on providing simplicity to the user, and in this thesis, we focus on the same idea, from the perspective that the user should be able to write a simple loopnest and have reasonable GPU primitives inferred from that code.

Other recent work has focused on developing higher-level DSLs which enable simple and fast prototyping of GPU kernels for AI workloads. Two of the most prominent are Triton and ThunderKittens. Triton provides a Python programming model based on block-level tensor operations and a compiler for tiled neural-network computations [26]. While significantly simpler than both CUDA and its corresponding DSLs, Triton has seen wide adoption in part because its simple abstraction allows for fast prototyping with competitive

performance to bare-metal CUDA. On the other hand, ThunderKittens takes a lower-level C++ approach. To support many of the complex APIs exposed on Hopper and later GPUs, ThunderKittens provides compact tile primitives that abstract away the TMA and WGMMMA while still allowing the programmer to easily manage synchronization and memory movement [24]. Building on Triton and ThunderKittens, TileLang has adopted some of the principles of Halide for low-level GPU DSLs, decoupling scheduling primitives like pipelining from the overall dataflow of the kernel being written [19, 28].

However, all of these systems, CuTe included, ask the programmer to think in tiles, requiring tile shapes and pipeline stages to be specified in the source code. As such, these systems require the programmer to reason about both advanced C++ template programming and low-level hardware constraints like register allocation. We attempt to further abstract this complexity away, going straight from loopnests to performant tile-based computations, choosing simplicity over generality.

3.2.2 Tensor Compilers and Languages

Another key direction for performant GPU programming has been the development of tensor compilers, which extract tensor programs from loop computations and lower them to specialized hardware. In general, tensor compilers operate by taking a loopnest represented in either a targeted IR or a general programming language and lowering it down to machine code, where the IR typically operates on large matrix operations (also termed tensor operations).

One direction in tensor compilers involves taking high-level loopnests and converting them into low-level machine code, with approaches like UNIT and TensorIR, using block-level and loop-level IRs and analysis to determine what tensor instruction sets can be used to lower the program [6, 29]. Other approaches like AMOS and STeP extend a similar approach to dataflow accelerators, providing native support for dynamic tensor operations and other hardware abstractions to map loopnests onto generic dataflow hardware [23, 34]. The other direction in tensor programs is low-level tensor IRs, like Graphene and Hidet, which instead of adopting high-level loopnest based analyses, directly map tensor computations to specific hardware execution resources and threads [5, 9]. These approaches are similar to the tile-based dataflows supported by CuTe, Triton, and ThunderKittens [17, 24, 26].

As a whole, these systems provide a useful abstraction for analyzing loopnests and matrix computations, but they either require low-level IR programming with specific tile-based analyses, or require rewriting and reordering loopnests entirely to support relevant hardware operations, which would be difficult to incorporate without restructuring existing BuildIt code. Additionally, the high-level tensor compilers do not support a wide variety of synchronization primitives, which is required for modern GPU kernels. In this thesis,

we explore whether we can adopt many of the ideas from these loopnest analyses, while still supporting synchronization primitives directly from native BuildIt loopnests.

3.3 Hand-Optimized Kernels

Finally, this thesis leans on the open literature of hand-written Hopper GEMM kernels, particularly the kernels provided in both open-source reproductions and the CUTLASS/CuTe libraries [15, 21, 22]. These kernels expose the key design decisions required for writing performant Hopper kernels, such as warp-specialization, proper pipelining of the TMA and WGMMA, and more, many of which we adopt in our own generated code.

In addition to generic GEMM kernels, specialized algorithms like Stream-K provide support for small matrix sizes, where occupancy can be limited across the GPU, which is a tradeoff that we discuss later in Chapter 5 [18]. Additionally, many modern kernels, like FlashInfer and FlashAttention, prioritize a methodology known as kernel fusion, where multiple large operations are combined, or fused, into a single kernel to minimize inter-operation communication latency [3, 4, 20, 32]. Although this thesis focuses primarily on implementing high-performance GEMM primitives in BuildIt, we recognize that many of the downstream use cases explicitly require kernel fusion, and we discuss how our approach can be extended to support fused kernels in Chapter 6.

Chapter 4

System Design and Implementation

In this thesis, we thus extend BuildIt to generate performant DSLs for modern GPUs, leveraging both general GPU features like shared memory and Hopper-specific features like the TMA and WGMMMA. At a high level, the final system takes a staged source program with loopnest-based GPU computation and emits performant CuTe code that uses shared memory, TMA-style tile movement, WGMMMA-style compute, and producer-consumer synchronization.

In the following sections, we discuss the implementation pieces required to reach that target.

4.1 Overview and Design Philosophy

One of the core design decisions of BuildIt is that the user should not have to write code for a specific hardware target, and should instead be able to use annotations to change the targeted hardware architecture. For example, while BuildIt can emit CUDA kernels by using the existing CUDA extraction pass with a `CUDA_KERNEL` annotation, the baseline code would still work on CPU if left unannotated.

We aim to maintain the same annotation-driven programming model in our own implementation of GPU matrix multiplication primitives, meaning that whether or not the user chooses to opt into using the TMA and WGMMMA should have no impact on code correctness. To maintain this abstraction, our implementation uses two additional annotations, `WGMMMA_OUTER` and `WGMMMA_INNER`, to mark the outer and inner loop of the matrix multiplication loopnest being lowered. These annotations can only be used within a `CUDA_KERNEL` annotation, ensuring backwards compatibility even when inferring a WGMMMA and TMA is not strictly possible.

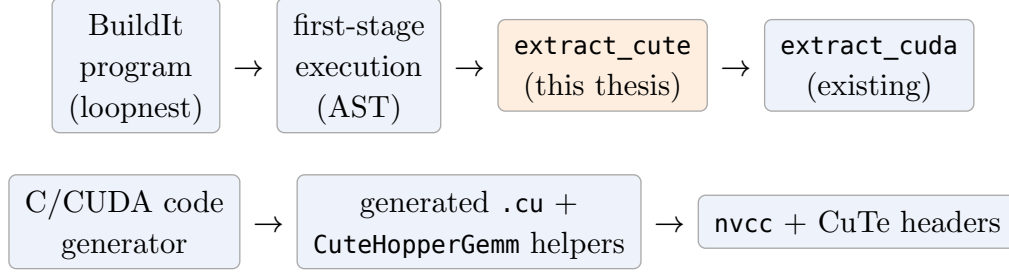


Figure 6: The compilation pipeline used with the new `extract_cute` pass. The new pass modifies the loopnest in the AST in-place for the existing `extract_cuda` pass to lower.

To implement the backwards compatibility mentioned above, we choose to design `extract_cute` as a preliminary AST analysis pass instead of a drop-in replacement for the current CUDA extraction pass. As shown in Figure 6, the `extract_cute` pass modifies the AST in-place, replacing only the annotated matmul subsection with relevant specialized calls to the templated helper file. All surrounding code, including all management of the CTAs and threads is deferred to the `extract_cuda` pass, ensuring that all prologue, epilogue, and kernel launch code is left untouched. As a result, the `extract_cute` pass works collaboratively with the `extract_cuda` pass to represent a wide range of possible kernels.

4.2 GPU Memory Buffers

The first step to enabling TMA and WGMMA use in BuildIt was to support various GPU memory buffers, such as shared memory. While the TMA and WGMMA explicitly use shared memory as part of the input and output arguments, BuildIt previously had no way to support these annotations, making adding support a prerequisite for the `extract_cute` pass.

In CUDA, to mark a variable as shared, a programmer typically uses the annotation `__shared__` before the data type, indicating to the CUDA compiler that this variable lives in the shared scratchpad and not in registers or on the host. However, this declaration differs from ordinary C++ type syntax because living on the shared scratchpad is inherently an attribute of the specific variable, not the type. For example, consider an integer array in shared memory. In terms of every other compiler analysis pass, the type of the variable is functionally still an integer array, as the location of the buffer does not change the type behavior. However, for this pass, we must be aware of which buffers live on shared memory versus global memory to properly coordinate data movement.

To implement this in BuildIt, we attach a `cuda_annotations` field to `block::var`, which lives as a separate attribute of each variable defined in the program. To provide user-side functionality for using this field, we add an `add_cuda_annotation` helper on each dynamic variable (`dyn_var`) so that users can attach GPU memory annotations natively in their

code. During CUDA emission, the code generator checks all CUDA annotations and prints them before the type, allowing BuildIt to generate declarations like those in Figure 7.

```
1 __shared__ half smem_a[BM * BK * QSIZE];
2 const __grid_constant__ CUTensorMap desc_a;
```

Figure 7: The `cuda_annotations` field allows the user to write code that supports memory annotations like `__shared__` and `__grid_constant__`.

Since the annotations live within the AST node for each variable, no existing passes modify them, making the CUDA annotations work out of box. However, the CUDA extraction pass still required small modifications.

Namely, shared memory buffers (and other GPU specific memory buffers) cannot be allocated on host-side processes, all declarations with CUDA annotations are hoisted from caller code into the generated kernel body, preventing the variables from being labeled as external and hence being passed as a kernel argument. Figure 8 shows a simple memory copy kernel implemented using a shared memory buffer, with the shared memory buffer being hoisted into the kernel as needed. As shown, the staged input is transformed into a CUDA kernel leveraging shared memory.

```
1 // Staged input
2 static void tile_copy(dyn_var<int*> src, dyn_var<int*> dst) {
3     dyn_var<int[512]> smem;
4     smem.add_cuda_annotation("__shared__");
5     builder::annotate(CUDA_KERNEL);
6     for (dyn_var<int> cta = 0; cta < 128; cta = cta + 1)
7         for (dyn_var<int> tid = 0; tid < 512; tid = tid + 1) {
8             smem[tid] = src[cta * 512 + tid];
9             dst[cta * 512 + tid] = smem[tid];
10        }
11 }
12
13 // Generated kernel
14 void __global__ cuda_kernel_0 (int* arg0, int* arg1) {
15     __shared__ int var3[512];
16     var3[threadIdx.x] = arg0[blockIdx.x * 512 + threadIdx.x];
17     arg1[blockIdx.x * 512 + threadIdx.x] = var3[threadIdx.x];
18 }
```

Figure 8: An example of a kernel implementing `memcpy` using shared memory, where the shared memory buffer is hoisted into the kernel.

4.3 Staged CuTe Helper Functions

In order to enable BuildIt support for lowering to CuTe, we next implement a series of staged helper hooks that use templating to help BuildIt generate code for constructing tile representations, invoking the TMA, and issuing WGMMA operations.

One key attribute of BuildIt that we wish to maintain is that the first stage of BuildIt compilation should not have to be aware of the specific requirements of any external

libraries. To maintain this invariant, we implement CuTe helper functions within BuildIt’s native type system as opaque function calls instead of custom AST nodes that the passes would have to reason about. Specifically, the generated tile computation does not become a custom IR node with a custom code generator, but instead, emits a plain call to a named device function, with the arguments encoding tile coordinates and problem sizes. This abstraction maintains the simplicity of the BuildIt compiler and enables us to strictly delineate all Hopper-specific behavior to an extraction pass, and not the core BuildIt functionality.

To implement these opaque function calls that BuildIt requires, we expose all CuTe-facing functionality through one templated device-side structure, `CuteHopperGemm`, described in Figure 9.

```

1  template <int TileM_, int TileN_, int TileK_,
2           int Stages_ = 2, int Threads_ = 128,
3           typename ElemA_ = cute::half_t,
4           typename ElemB_ = cute::half_t,
5           typename ElemC_ = float,
6           cute::GMMMA::Major MajorA_ = cute::GMMMA::Major::K,
7           cute::GMMMA::Major MajorB_ = cute::GMMMA::Major::MN,
8           typename StrideA_ = cute::Stride<int, cute::_1>,
9           typename StrideB_ = cute::Stride<cute::_1, int>,
10          typename ElemOut_ = ElemC_,
11          bool COutColMajor_ = false>
12
13  struct CuteHopperGemm { ... };

```

Figure 9: The templated CuTe helper structure used by BuildIt enables the pass to emit all relevant matrix attributes, like types, memory layout and pipelining parameters as compile-time templates.

As shown in Figure 9, `CuteHopperGemm` provides all parameters required for describing a full matrix multiplication, which are the data types, memory layout, and pipelining parameters. Apart from these attributes, all other relevant algorithmic details from the GEMM can be derived from the templated parameters. For example, we build 128 byte swizzled shared memory layouts by feeding the provided majority parameters through the canonical CuTe GMMMA layouts. Similarly, we use the element types, tile shapes, and memory layouts to select canonical WGMMA and TMA atoms using CuTe’s native `ss_op_selector` and `make_tma_copy`.

Beyond inferring the relevant parameters and layouts for the GEMM, `CuteHopperGemm` also provides the device-side methods that BuildIt calls in generated code. The key helper used by all generated code is `gemm_tile`, which computes a single $\text{TileM} \times \text{TileN}$ output tile using a non-warp specialized, one warpgroup pipeline. A warp-specialized version of `gemm_tile` is described in Section 4.6. As depicted in Figure 10, `gemm_tile` initializes the pipeline of TMA loads, before looping over batches in the K-direction, simultaneously issuing WGMMA operations for earlier pipeline stages and queuing new TMA loads. With $\text{TileK} = 64$, each batch is four `wgmma` instructions bracketed by accumulator fences. Once

all K-stages are complete, we finally load the output tile from registers back to global memory.

```

1  static __device__ __forceinline__
2  void gemm_tile(TmaA const& tma_a, TmaB const& tma_b,
3                ElemA* smem_A, ElemB* smem_B, uint64_t* mbar,
4                ElemOut* C, int cta_m, int cta_n, int M, int N, int K) {
5
6      int num_tiles_k = K / TileK;
7      AccumTensor rC;  cute::clear(rC);
8
9      // initialize barriers
10     if (threadIdx.x == 0)
11         for (int s = 0; s < Stages; ++s)
12             cute::initialize_barrier(mbar[s], 1);
13
14     // initialize the pipeline
15     __syncthreads();
16     if (threadIdx.x == 0)
17         for (int s = 0; s < min(num_tiles_k, Stages); ++s) {
18             cute::set_barrier_transaction_bytes(mbar[s], tx_bytes);
19             tma_load_A(tma_a, mbar[s], smem_A, s, cta_m, M, K);
20             tma_load_B(tma_b, mbar[s], smem_B, s, cta_n, N, K);
21         }
22
23     __syncthreads();
24     for (int k = 0; k < num_tiles_k; ++k) {
25         // wait for pipeline stage to finish loading
26         int stage = k % Stages, phase = (k / Stages) & 1;
27         cute::wait_barrier(mbar[stage], phase);
28         __syncthreads();
29
30         // queue up WGMMMA + next set of TMA loads
31         wgmma(TiledMMA{}, smem_A, smem_B, rC, stage, threadIdx.x);
32         if (k + Stages < num_tiles_k && threadIdx.x == 0) {
33             cute::set_barrier_transaction_bytes(mbar[stage], tx_bytes);
34             tma_load_A(tma_a, mbar[stage], smem_A, k + Stages, cta_m, M, K);
35             tma_load_B(tma_b, mbar[stage], smem_B, k + Stages, cta_n, N, K);
36         }
37         __syncthreads();
38     }
39
40     // store output tile to global memory
41     cute::copy(rC, thr_mma.partition_C(gC_tile));
42 }

```

Figure 10: The generic single-warpgroup `gemm_tile` implementation, shortened for clarity, which creates the TMA + WGMMMA pipeline before iterating over the K-dimension to calculate the values of one output tile.

4.4 The `extract_cute` Loopnest Analysis

In order to use the templated helpers defined in `CuteHopperGemm`, the final key part of our implementation is a new analysis pass, `extract_cute`, which lowers native `BuildIt` loopnests to calls to the `CuteHopperGemm` helpers. This section covers all loopnest analyses performed by `extract_cute`, which entails **annotating relevant loops**, **extracting loop bounds**, **enforcing hardware invariants**, and **inferring operand memory types/layouts**.

4.4.1 The Annotated Input Pattern

To begin, the `extract_cute` pass consumes two new annotations in order to represent the two different types of loops present in a typical matrix multiplication loopnest. Specifically, when we matrix multiply a $M \times K$ matrix by a $K \times N$ matrix, the M and N dimensions are present in the output $M \times N$ matrix as well. On the other hand, the K dimension is absorbed, and hence becomes an inner loop in the loopnest. As a result, we mark the outer loops with `WGMMMA_OUTER`, and the inner K -loop with `WGMMMA_INNER`, as depicted in Figure 11.

```
1  struct half_t {
2      static constexpr const char* type_name = "cute::half_t";
3      friend float operator*(const half_t&, const half_t&);
4  };
5
6  static void matmul(dyn_var<half_t*> A, dyn_var<half_t*> B, dyn_var<float*> C,
7                    dyn_var<int> M, dyn_var<int> N, dyn_var<int> K) {
8      //outer kernel loops
9      builder::annotate(CUDA_KERNEL);
10     for (dyn_var<int> cta = 0; cta < (M/256)*(N/128); cta = cta + 1) {
11         for (dyn_var<int> tid = 0; tid < 128; tid = tid + 1) {
12             dyn_var<int> i_n = (cta/(N/128));
13             dyn_var<int> j_n = (cta%(N/128));
14
15             // outer M and N loops
16             builder::annotate(WGMMMA_OUTER);
17             for (dyn_var<int> i = i_n*256; i < i_n*256 + 256; i = i + 1) {
18                 for (dyn_var<int> j = j_n*128 + tid; j < j_n*128 + 128; j = j + 128) {
19
20                     //inner K loop
21                     builder::annotate(WGMMMA_INNER);
22                     for (dyn_var<int> k = 0; k < K; k = k + 1)
23                         C[i * N + j] = C[i * N + j] + A[i * K + k] * B[k * N + j];
24                 }
25             }
26         }
27     }
28 }
```

Figure 11: The loopnest implementation of a GEMM using the `WGMMMA_INNER` and `WGMMMA_OUTER` annotations. Each CTA processes a 256x128 block of the output matrix, which is recovered by the pass using loop bounds.

One key attribute of this program is that despite being lowered into WGMMMA and TMA operations by the pass, it is functionally correct even without the lowerings. While performance would change, with only the `CUDA_KERNEL` directive, BuildIt would be able to lower this to a working GEMM kernel, demonstrating backwards compatibility.

Another key attribute of this program is that the user does not have to manually specify tiling constraints to the pass. By analyzing the loop structure and bounds, the pass can infer what pieces of the output matrix are owned by each CTA, essentially rasterizing the output by CTA. As part of this rasterization, the index expressions used in the inner loop (e.g. $i \cdot K + k$ and $i \cdot N + j$) are used to define memory layouts. By allowing the pass to intelligently determine tiling and memory parameters from the loopnest, rather than a separate scheduling language, we ensure that the entire source BuildIt program remains

concise and readable. We elaborate on the specifics of these loopnest extractions in the next sections.

4.4.2 Extracting Loop Bounds

The first part of extracting relevant parameters from the loopnest is extracting the loop bounds, which allow us to determine what chunk of the output matrix each CTA handles. We perform this by implementing a small affine analyzer, which operates over the `cta` and `tid` variables.

To design this affine analyzer, we derive a simple expression format for loopnest bounds, where each term t is a division/modulus expression over `cta` and each term c_t , c_{tid} , or c_0 is an integer coefficient:

$$\text{expr} = \sum_t c_t \cdot t + c_{tid} \cdot \text{tid} + c_0$$

For example, applied to Figure 11, the lower bound of the `i` loop decomposes to the single term $t = \text{cta}/(\text{N}/128)$ with $c_t = 256$, while the lower bound of the `j` loop decomposes to $t = \text{cta} \% (\text{N}/128)$ with coefficients $c_t = 128$ and $c_{tid} = 1$.

Table 1: Expression tree walk rules used by the affine analyzer to calculate the relevant constants required to transform each loop bound into the standardized expression format described above.

Expression	Affine Analysis Result
integer constant c	Empty expression with $c_0 = c$
<code>tid</code>	Empty expression with $c_{tid} = 1$
<code>cta</code>	Empty expression with $t = \text{cta}$, $c_t = 1$
<code>cta / D</code> , <code>cta % D</code>	Opaque term with $t = \text{cta} / D$ or $t = \text{cta} \% D$, $c_t = 1$, with <code>D</code> kept as an AST reference
Nested div/mod over <code>cta</code> terms	Similar to above, opaque term, with relevant loop-independent <code>D</code> kept as AST reference
$e_1 \pm e_2$	Merge expressions and all relevant c
$c_e \cdot e$ (loop-independent c_e)	Scale all coefficients $c = c \cdot c_e$
Anything else involving <code>cta/tid</code>	Reject the loopnest

We derive this expression by walking the expression tree for each loop bound, following the rules derived in Table 1. All rules are fairly standard, outside of the rule to calculate the `cta` based expression t . For each expression t , we treat all nested division and modulo expressions over `cta` terms as opaque terms involving loop-independent constants `D`, which are kept as references to the base BuildIt AST. This delineation allows us to represent all nested `cta` expressions without special cases while still ensuring that later passes which

modify the AST correspondingly modify all variables in loop bounds, preserving code correctness.

We provide another example of loop bound extraction based on Figure 12, where like the example from Figure 11, the opaque terms allow us to extract complex loopnest bounds. In Figure 12, the loopnest walks the output matrix in groups of 8 CTAs, keeping consecutive CTAs close together.

```

1 // CTAs traverse the output in groups of 8 rows
2 for (dyn_var<int> i = ((cta/(8*(N/128)))*8 + (cta%(8*(N/128)))%8)*256;
3     i < ((cta/(8*(N/128)))*8 + (cta%(8*(N/128)))%8)*256 + 256;
4     i = i + 1) {
5
6     for (dyn_var<int> j = ((cta%(8*(N/128)))/8)*128 + tid;
7         j < ((cta%(8*(N/128)))/8)*128 + 128;
8         j = j + 128) {

```

Figure 12: In this example, the loop bounds are complex, as it represents sets of 8 CTAs walking the output matrix in groups. Using opaque terms, the affine analyzer is still able to extract relevant loop bounds.

Just like with the Figure 11 loopnest, the first thing the affine analyzer does is walk over the expression tree for each bound. Here, for the *i* loop, the analyzer first sees two opaque expressions involving *cta*, which are $t_1 = cta/(8*(N/128))$ and $t_2 = cta\%(8*(N/128))$, each with $c_t = 1$. Then, since t_2 is modulated by another modulo 8 operation, the analyzer continues to just treat t_2 as a larger opaque term, changing $t_2 = (cta\%(8*(N/128)))\%8$. Using these opaque terms, the affine analyzer now sees the loop bound as $(t_1 * 8 + t_2) * 256$, which by decomposing the multiplications results in $c_{t1} = 8*256 = 2048$ and $c_{t2} = 256$. Similarly, for the *j* loop, the lower bound can be represented using opaque terms by the analyzer as $t = (cta\%(8*(N/128)))/8$, with $c_t = 128$ and $c_{tid} = 1$.

As a whole, we use the examples of both Figure 11 and Figure 12 to illustrate that the use of opaque terms allows the affine analyzer to represent arbitrarily complex loop bounds without needing to understand the underlying arithmetic. Instead, the underlying arithmetic can be deferred to later analyses, which leverage the general equation form created by the affine pass.

4.4.3 Enforcing Loop Invariants

Once the affine analyzer extracts expressions for each loop bound, we subtract the analyzed lower bound from the upper bound for each loop. This result cancels matching terms, yielding the loop's span as a constant. For example, when analyzing Figure 11, the pass would thus learn that the *i* loop spans 256 elements, whereas the *j* loop only spans 128. Using these spans, the pass determines the size and location of the output matrix chunk that each CTA owns. Additionally, if the bounds for a loop contain a *tid* term, the pass

considers that to be *thread-shared*, enforcing that the threads cover the entire span of the loop.

The calculated spans are also useful for enforcing a number of hardware invariants, specifically with regard to the WGMMMA operation size. Because each WGMMMA operation operates on exactly a $64 \times N$ tile of the output matrix (where $N = 128, 256$ is selected based on warpgroup count), we mandate that each CTA must operate on a contiguous tile that is some whole multiple of this base hardware tile size. A chunk smaller than the tile is reported as an error rather than silently generating a kernel that computes incorrectly. Currently, the pass also requires that the total problem size divides evenly by the tile size, and remainder handling is discussed as future work in Chapter 6. We also enforce that the number of threads is a multiple of 128, as WGMMMA requires full warpgroups of 128 threads to execute in lockstep.

After this stage, we thus enforce the invariant that each CTA processes a $m_c \times n_c$ block of the output matrix, where the output tile can be broken down into $(m_c/64) \times (n_c/128)$ WGMMMA tiles.

4.4.4 Inferring Operand Types

The pass first infers operand type directly from staged types, as shown in Figure 13. Staged types are a natural fit for BuildIt because the first stage never directly compiles GPU code, allowing lightweight marker structs that provide a thin wrapper over type semantics. For example, in Figure 13, the `type_name` member describes the exact type emitted in generated code. The multiply operator indicates that two FP16 inputs multiply to FP32, matching WGMMMA type semantics without needing runtime information.

```
1 struct half_t {
2     static constexpr const char* type_name = "cute::half_t";
3     friend float operator*(const half_t&, const half_t&);
4 };
```

Figure 13: A sample definition of the `half_t` marker struct, which contains the actual type name of `cute::half_t` and a description of FP16 multiplication semantics.

4.4.4.1 Mixed Precision Output Matrices

```
1 struct bfloat16_t {
2     static constexpr const char* type_name = "cute::bfloat16_t";
3     friend float operator*(const bfloat16_t&, const bfloat16_t&);
4     friend bfloat16_t operator+(const bfloat16_t&, float);
5 };
```

Figure 14: A definition of a BF16 type that supports mixed precision output matrices, where accumulation is performed in FP32, but output is stored in BF16.

Especially in machine learning workflows, many data types like BFloat16 (BF16) operate with mixed precision in the output. The `extract_cute` pass supports mixed output precision out-of-box by separating the `ElemC` accumulator type from the `ElemOut` output type. This dichotomy can also be mirrored in the type declaration. For example, given a BF16 matrix multiply that accumulates in FP32 for numerical stability but outputs in BF16, the user can overload the BF16 multiply operation to output a float (indicating FP32 accumulation), but simultaneously update mixed-precision BF16+FP32 adds to output in BF16, matching the desired BF16 output. An example of how this is implemented is illustrated in Figure 14.

4.4.5 Inferring Operand Layouts

Operand memory layouts are extracted by analyzing the index for each operand matrix and determining the coefficients of each loop variable, which are represented as AST expressions. For example, for `A[i*K + k]`, the coefficient of `i` would be `K` and the coefficient of `k` would be `1`. The dimension with unit stride determines the operand’s major axis, or the fastest moving-dimension in the loopnest. The operand is thus considered K-major when the contraction index `k` is contiguous, and MN-major when the output index `m` or `n` is. Using each operand’s major axis, `BuildIt` can thus instantiate `CuteHopperGemm` with the right parameters for swizzling, WGMMA layouts, and TMA descriptors, which are created using the extracted strides.

The output matrix is analyzed using the same rule. A unit stride in `j` (`C[i*N + j]`) is considered row-major (or N-major), and a unit stride in `i` is column-major (or M-major). Unlike the input matrices, `C` is never read through the TMA and WGMMA, instead living in registers, so the memory layout of `C` is communicated through `CuteHopperGemm`’s `COutColMajor` flag rather than tensor layouts and strides. However, the layout of the output matrix still impacts performance significantly, as described in Chapter 5.

4.4.5.1 Batched Matrix Multiplication

To detect batching in the GEMM, and thus in the operand layout, the `extract_cute` pass detects a loop between the `CUDA_KERNEL` annotated loops and `WGMMA_OUTER` annotated loops that the GEMM is invariant to (aside from shifting the GEMM base address by some fixed per-operand stride). This loop is considered to be a batch loop, and is folded into the TMA descriptors, as a third dimension in the TMA descriptor. We choose this interface as the TMA natively supports higher dimensional accesses and because otherwise, we would have to generate a separate TMA descriptor per batch element.

4.5 The `extract_cute` Output Pass

Once the kernel is fully processed by the affine and loopnest analyses, the loopnest is converted to working CUDA code by the output pass of `extract_cute`, which we describe in the following subsections.

4.5.1 Emitting the Kernel

To output the kernel produced by the `extract_cute` pass, there are two key components: a **textual preamble** of helper definitions, and an **in-place AST rewrite**.

In the preamble, the `extract_cute` pass instantiates the `CuteHopperGemm` helper with the inferred types, layouts, and strides from the loopnest analysis. Using the functionalities within the helper, the preamble defines type aliases (such as specific TMA descriptors), tile-size constants, and a one line device function, `buildit_gemm_tile`, that wraps the `gemm_tile` helper provided by `CuteHopperGemm`. In addition, to prevent aliasing between the types from multiple inferred GEMMs, every identifier is generated with a unique suffix, like `Gemm_t_0`.

Inside the AST, the pass takes the $m_c \times n_c$ output block processed by each CTA and divides it into a new loopnest over 64×128 WGMMA tiles. Each iteration of the loopnest then becomes a single call to `buildit_gemm_tile`, which executes the entire K-loop for a given WGMMA tile after being provided with TMA descriptors, problem sizes, and shared memory pointers. Figure 15 shows the result of kernel emission for Figure 11, abridged for clarity. The full length of the kernel is only around 50 lines.

```

1 // helper inferred with loopnest parameters
2 using Gemm_t_0 = CuteHopperGemm<64, 128, 64, 2, 128,
3     cute::half_t, cute::half_t, float,
4     cute::GMMA::Major::K, cute::GMMA::Major::MN,
5     cute::Stride<int, cute::_1>, cute::Stride<cute::_1, int>>;
6
7 // type aliases and tile-size constants omitted for clarity
8
9 void __global__ cuda_kernel_0 (TmaA_0 const __grid_constant__ arg0,
10     TmaB_0 const __grid_constant__ arg1,
11     float* arg2, int arg3, int arg4, int arg5) {
12
13     // initialize buffers
14     extern __shared__ __align__(128) char buildit_smem[];
15     ElemA_0* smem_A = (ElemA_0*)buildit_smem;
16     ElemB_0* smem_B = /* offset past A stages */;
17     uint64_t* mbar = /* offset past B stages */;
18
19     // loop over WGMMMA tiles
20     for (int mt = 0; mt < 4; mt = mt + 1)
21         for (int nt = 0; nt < 1; nt = nt + 1)
22             buildit_gemm_tile_0(arg0, arg1, smem_A, smem_B, mbar, arg2,
23                 blockIdx.x / (arg3 / 128) * 4 + mt, blockIdx.x % (arg3 / 128) + nt,
24                 arg4, arg3, arg5);
25 }
26
27 // host-side declaration
28 void matmul (cute::half_t* arg0, cute::half_t* arg1, float* arg2,
29     int arg3, int arg4, int arg5) {
30
31     // initialize TMA
32     TmaA_0 tma_a = make_tma_copy(SM90_TMA_LOAD{},
33         make_tensor(make_gmem_ptr(arg0),
34             make_layout(make_shape(arg3, arg5),
35                 make_stride(arg5, _1{}))),
36         SmemLayoutA_0{}{_, _, Int<0>{}});
37
38     TmaB_0 tma_b = /* symmetric, with make_stride(_1{}, arg4) */;
39
40     // kernel invocation
41     cudaFuncSetAttribute((void*)cuda_kernel_0,
42         cudaFuncAttributeMaxDynamicSharedMemorySize, kSmemBytes_0);
43     cuda_kernel_0<<<arg3 / 256 * (arg4 / 128), 128, kSmemBytes_0>>>(
44         tma_a, tma_b, arg2, arg4, arg3, arg5);
45     cudaDeviceSynchronize();
46 }

```

Figure 15: The emitted kernel for the loopnest in Figure 11 instantiates `CuteHopperGemm` with the inferred loopnest parameter, walks the CTA’s four 64×128 tiles, and builds the TMA descriptors.

One note is that instead of producing memory management helpers like `cudaMalloc`, the pass does not make any assumptions about the location of the data buffers, instead giving the programmer the flexibility to place buffers as desired. The pass only outputs the minimal scaffolding required to invoke the TMA and WGMMMA, which amounts to the shared memory layouts and TMA descriptors.

However, the pass does reason about certain shared memory constraints because with `TileK = 64`, the pipeline’s shared-memory footprint is already 48 KB for two stages, which

is at the limit of what CUDA permits statically. As a result, the pass always emits a single `extern __shared__` dynamic allocation, carves the A stages, B stages, and barrier array out of it at 128-byte alignment, and routes the byte count through `cudaFuncSetAttribute` before the launch.

4.5.2 CUDA Extraction Changes

Beyond the `extract_cute` pass, we also implement a few small modifications to the `extract_cuda_from` pass required for producing GEMM kernels using WGMMMA and TMA. The first modification is allowing the CUDA pass to reason about changing the amount of shared memory or registers used. Since the existing pass did not reason about shared memory at all, it had no need to reason about the amount of shared memory or registers. However, for a standard WGMMMA+TMA pipeline, with only two stages and a tile size of 64 in the K direction, the shared memory footprint already reaches the 48 KB static limit allowed per block. However, each SM can support up to 227 KB of dynamic shared memory, so we pass attributes from the CuTe pass to the CUDA pass that expand the amount of shared memory using the `cudaFuncAttributeMaxDynamicSharedMemorySize` attribute. Similarly to shared memory, since warp specialization requires partitioning registers unequally between warpgroups, as mentioned in Chapter 2, the CuTe pass signals to the CUDA pass that it must emit a `__launch_bounds__` attribute to properly set register use. Finally, we add custom kernel parameter metadata to the TMA descriptors that the CuTe pass emits, which must be passed using `const __grid_constant__`, as TMA descriptors can only be defined using host side machinery.

4.6 Warp Specialization

The main optimization we support beyond the core pass logic is warp specialization, which users can implicitly request by increasing the amount of threads per CTA. Because the `extract_cute` pass cannot impact CUDA kernel code outside of the targeted WGMMMA+TMA loopnest, the pass cannot change the number of threads provided, and thus must rely on the user to provide multiple warpgroups if warp specialization is desired. In general, we require the `tid` loop bound to be a multiple of 128 when the `extract_cute` pass is used, providing $W = T / 128$ warpgroups, where T is the `tid` loop bound. For $W = 1$, the pass utilizes the single-warpgroup path, but when $W \geq 2$, the pass uses warpgroup 0 as a dedicated TMA producer, whereas warpgroups 1 through $W - 1$ become consumers. An example of the source required to invoke warp specialization is provided in Figure 16, where the only change from before is the number of threads used.

```

1  builder::annotate(CUDA_KERNEL);
2  for (dyn_var<int> cta = 0; cta < (M/128)*(N/256); cta = cta + 1) {
3      for (dyn_var<int> tid = 0; tid < 384; tid = tid + 1) { // 3 warpgroups
4
5          dyn_var<int> i_n = (cta/(N/256));
6          dyn_var<int> j_n = (cta%(N/256));
7          builder::annotate(WGMMA_OUTER);
8          for (dyn_var<int> i = i_n*128; i < i_n*128 + 128; i = i + 1) {
9              for (dyn_var<int> j = j_n*256 + tid; j < j_n*256 + 256; j = j + 256) {
10
11                  builder::annotate(WGMMA_INNER);
12                  for (dyn_var<int> k = 0; k < K; k = k + 1)
13                      C[i * N + j] = C[i * N + j] + A[i * K + k] * B[k * N + j];
14              }
15          }
16      }
17  }

```

Figure 16: To invoke warp specialization, the only change required in the input loopnest is to change the number of threads to support multiple warpgroups of 128 threads.

Instead of calling the generic `gemm_tile` helper described in Figure 10, warp specialized kernels call a separate helper, `gemm_tile_ws`, which is shown in Figure 17. There is one key difference in the implementation. Instead of one barrier per pipeline stage, which is sufficient to synchronize the single warpgroup case, the warp specialized version uses a pair of barriers per pipeline stage in the warp specialized helper to communicate across warpgroups. Specifically, each stage contains both a `full` and an `empty` barrier. The `full` barrier is used by the producer warpgroup to signal to the consumer warpgroup that the TMA has finished its memory load, indicating that the pipeline stage is ready for a WGMMA invocation. Similarly, the `empty` barrier is used by the consumer to signal to the producer that the WGMMA operation has completed, making the pipeline stage ready for a new TMA load.

As mentioned before, warp specialization requires changing the number of registers used per thread, which is performed using the built-in CUTLASS `setmaxnreg` wrappers. Using the warpgroup count, we calculate the number of registers used on producers and consumers, which for example, becomes 40 for the producer and 232 per consumer for $W = 3$. This register rebalancing also allows the warp specialized kernel to use a large WGMMA with a 64×256 output tile, further amortizing the WGMMA launch overhead.

```

1 // producer warpgroup
2 if (threadIdx.x < 128) {
3     cutlass::arch::warpgroup_reg_dealloc<40>(); // reduce register allocation
4
5     // launch TMA from thread 0 only
6     if (threadIdx.x == 0)
7         for (int k = 0; k < num_tiles_k; ++k) {
8             int s = k % Stages, wave = k / Stages;
9
10            // wait for pipeline stage to be empty
11            if (wave > 0)
12                cute::wait_barrier(empty[s], (wave - 1) & 1);
13
14            // launch TMA loads
15            cute::set_barrier_transaction_bytes(full[s], tx_bytes);
16            tma_load_A(tma_a, full[s], smem_A, k, cta_m, M, K);
17            tma_load_B(tma_b, full[s], smem_B, k, cta_n, N, K);
18        }
19 }
20
21 // consumer warpgroups
22 else {
23     cutlass::arch::warpgroup_reg_alloc<232>(); // increase register allocation
24     int ctid = threadIdx.x - 128;
25     AccumTensor rC; cute::clear(rC);
26
27     // launch WGMMMA from all threads
28     for (int k = 0; k < num_tiles_k; ++k) {
29         int s = k % Stages, wave = k / Stages;
30
31         // wait for pipeline stage to fill
32         cute::wait_barrier(full[s], wave & 1);
33         cute::warpgroup_fence_operand(rC);
34         cute::warpgroup_arrive();
35
36         // execute WGMMMA operation
37         cute::gemm(mma, thr_mma.partition_A(sA(_, _, s)),
38                  thr_mma.partition_B(sB(_, _, s)), rC);
39         cute::warpgroup_commit_batch();
40         cute::warpgroup_wait<0>();
41         cute::warpgroup_fence_operand(rC);
42         cute::arrive_barrier(empty[s]);
43     }
44
45     // copy output tile
46     cute::copy(rC, TiledMMA{}.get_slice(ctid).partition_C(gC_tile));
47 }
48 __syncthreads();

```

Figure 17: The generic warp-specialized `gemm_tile_ws` implementation, shortened for clarity, which creates the TMA + WGMMMA pipeline across separate producer and consumer warpgroups. Register counts are calculated based on the warpgroup count, in this case for $W = 3$.

We note that using `ctid` across two consumer warpgroups is correct because we also use CuTe to natively tile over two WGMMMA output tiles in the M direction when using warp specialization. As a result, CuTe reads `ctid` and allocates one output tile to one warpgroup, meaning the effective output tile size per `gemm_tile_ws` implementation is 128×256 .

Chapter 5

Evaluation

In this chapter, we evaluate our BuildIt CuTe extension along three axes:

1. **Correctness (Section 5.2)**
2. **Programmability (Section 5.3)**
3. **Performance (Section 5.4)**

We finish with a discussion of limitations in Section 5.5.

5.1 Experimental Setup

Since we do not have access to an H100 development server, we perform all development on a V100 server with 80 Broadwell-generation Intel Xeon CPU cores and the CUDA development platform. We run BuildIt, .cu file generation, compilation with `nvcc -arch=sm_90a`, and CPU reference results locally, cross-compiling for Hopper generation GPUs. For all correctness and performance benchmarks, we run them on rented H100 instances through the Modal cloud platform. Since we do not own the instances, and thus cannot quiesce across sessions, we run all comparisons in this chapter within a single session to minimize performance deltas. For example, while the baseline cuBLAS run varied between 634 and 761 TFLOP/s median for $4096 \times 4096 \times 4096$ GEMM across multiple sessions due to thermal and clocking differences, the variance within each session was small.

To measure timing, we place CUDA events around each kernel invocation and report the median of 20 iterations after 5 warmup iterations. All input buffers are allocated once and filled deterministically on the device, so the generated kernels and cuBLAS always see identical inputs. To save H100 time, all deterministic input buffers are pre-computed on the Xeon development server.

5.2 Correctness Validation

We validate correctness against reference results computed on the CPU using double precision and on the H100 using cuBLAS. Both comparisons use the relative Frobenius error (RFE):

$$\frac{\|C_{\text{test}} - C_{\text{ref}}\|_F}{\|C_{\text{ref}}\|_F}$$

with a pass threshold of 10^{-2} , where the $\|C\|_F$ refers to the Frobenius norm of a matrix

$$\|C\|_F = \sqrt{\sum_{i=1}^n \sum_{j=1}^n |C_{ij}|^2}$$

Using this equation, the relative Frobenius error is functionally a normalized version of the root-mean-squared error (RMSE), where the ratio reduces to the RMSE divided by the root-mean-squared norm of the matrix.

Table 2: We write a variety of kernels with different compute setups, memory layouts, and problem sizes, validating each configuration against both a double-precision CPU reference and cuBLAS (CPU reference numbers not shown, but within epsilon). We find that all FP32 cases are within epsilon compared to cuBLAS, and BF16 kernels have small, but expected numerical imprecision.

Configuration	Problem Sizes	cuBLAS RFE
Baseline config: Row-major, FP16, one warpgroup	$256 \times 128 \times 64$, $512 \times 512 \times 512$, $768 \times 384 \times 256$, $256 \times 512 \times 128$, $1024 \times 1024 \times 1024$, $4096 \times 4096 \times 4096$	0.0
BF16 inputs	$512 \times 512 \times 512$, $768 \times 384 \times 256$	0.0
M-major A	$512 \times 512 \times 512$, $768 \times 384 \times 256$	0.0
K-major B	$512 \times 512 \times 512$, $768 \times 384 \times 256$	0.0
Batched	$4 \times 512 \times 512 \times 512$, $3 \times 768 \times 384 \times 256$	0.0
Warp-specialized (3 warpgroups)	$512 \times 512 \times 512$, $1024 \times 1024 \times 1024$, $2048 \times 2048 \times 2048$, $4096 \times 4096 \times 4096$	0.0
Warp-specialized, column-major C	$512 \times 512 \times 512$, $1024 \times 1024 \times 1024$, $2048 \times 2048 \times 2048$, $4096 \times 4096 \times 4096$	0.0
Warp-specialized, BF16 output	$4096 \times 4096 \times 4096$	1.644×10^{-3}
1-D CTA raster	$512 \times 512 \times 512$, $1024 \times 1024 \times 1024$, $2048 \times 2048 \times 2048$, $4096 \times 4096 \times 4096$	0.0

We present a suite of correctness tests against randomly generated input matrices in Table 2, covering a wide variety of input configurations, memory layouts, data types, and problem sizes. For each FP32-output configuration, the BuildIt produced kernel matched cuBLAS within epsilon, with zero RFE. All CPU references are also within epsilon. By construction, the BF16 output configuration is not as precise as the FP32 baseline, as

down-converting to BF16 involves a loss in precision, but the error still remains at a small value of 1.644×10^{-3} , validating correctness of the generated kernels.

5.3 Programmability

Table 3: Using BuildIt, a user can write a simple staged GEMM loopnest using as few as 15 lines of code, which generates 51 lines of performant, readable GEMM source code. In comparison, a hand-written kernel with CuTe or CUDA takes around 400 lines of code.

Code Artifact	Lines	Written by
Staged GEMM source	15	DSL user
BuildIt generated GEMM source	51	generated
CuteHopperGemm helper header (shared)	574	this thesis
extract_cute analysis pass (shared)	885	this thesis
hand-written Hopper GEMM kernel [21]	≈ 400	expert dev
CUTLASS warp-specialized GEMM kernel [12]	≈ 400	expert dev

We quantify the programmability of the BuildIt CuTe in terms of lines of code required in Table 3. With the BuildIt WGMMA+TMA functionality, the user can write as few as 15 lines of staged loopnest code to produce a working kernel, requiring no GPU constructs apart from the `CUDA_KERNEL`, `WGMMA_OUTER`, and `WGMMA_INNER` annotations. Even after BuildIt compilation, the source code only remains 51 lines of easily readable code, as presented in Chapter 4. Instead of exposing CUDA complexity to the user, it lies in abstract helpers that we write and which are shared across all generated source code, compared to the approximately 400 lines of CUDA/PTX/CuTe required by hand-written kernels [15, 21].

Another key observation is that all configuration changes described in Section 5.2 are minor, requiring at most a few lines of code change, as described in Table 4.

Table 4: Relative to the baseline configuration in Figure 11, every GEMM variant changes minimally.

GEMM Config Change	Corresponding Code Edit
BF16 inputs	(half_t $\rightarrow$ bfloat16_t)
M-major A matrix	A[i*K+k] $\rightarrow$ A[k*M+i]
K-major B matrix	B[k*N+j] $\rightarrow$ B[j*K+k]
M-major C matrix	C[i*N+j] $\rightarrow$ C[j*M+i]
BF16 output	output pointer type, mixed precision addition operator
batched (L matrices)	add batch loop and indexing
grouped L2 raster	nested div/mod arithmetic in loop bounds
warp specialization ($W = 3$)	increase thread count from 128 $\rightarrow$ 384

5.4 Performance

In this section, we demonstrate that the BuildIt CuTe pass can be used out of box to write flexible and performant kernels. Starting from a naive loopnest, we walk through a series of small kernel optimizations like those described in Section 5.3. For each optimization, we highlight three features:

1. **the specific code change made**, in terms of the loopnest as written,
2. **the downstream kernel change**, in terms of disassembled SASS assembly, and
3. **the measured performance impact** compared to the previous kernel iteration

Following Section 5.1, all head-to-head numbers come from a single H100 session, and performance numbers are provided as utilized FLOP/s and percentages of the cuBLAS baseline performance. As a guide for which kernel changes matter on Hopper GPUs, we lean on the open literature on hand-written GEMM optimization [15, 21, 22].

5.4.1 Baseline Kernel

We start with a simple baseline that consists of a loopnest with one thread per output element and no annotations beyond `CUDA_KERNEL`. The first change we make is turning the `extract_cute` pass on by adding the `WGMMMA_OUTER` and `WGMMMA_INNER` annotations. As a simple first kernel, we write the loopnest in a banded form, where each CTA processes a 1D band of the output matrix of width 256 that spans the full N dimension, as shown in Figure 18.


```

1 // kernel launch bounds
2 builder::annotate(CUDA_KERNEL);
3 for (dyn_var<int> cta = 0; cta < M/256; cta = cta + 1) {
4     for (dyn_var<int> tid = 0; tid < 128; tid = tid + 1) {
5
6         // output matrix bands
7         builder::annotate(WGMMA_OUTER);
8         for (dyn_var<int> i = cta*256; i < cta*256 + 256; i = i + 1) {
9             for (dyn_var<int> j = tid; j < N; j = j + 128) {
10
11                 // inner K loop
12                 builder::annotate(WGMMA_INNER);
13                 for (dyn_var<int> k = 0; k < K; k = k + 1)
14                     C[i * N + j] = C[i * N + j] + A[i * K + k] * B[k * N + j];
15             }
16         }
17     }
18 }

```

Figure 18: The baseline kernel for performance benchmarking has each CTA process one band of the output matrix of width 256.

With these changes, at a problem size of $4096 \times 4096 \times 4096$, throughput rises from 5.0 to 22.4 TFLOP/s, a significant increase even though the new kernel only achieves an occupancy of 16/132 SMs. The biggest change is that unlike the naive kernel, which exhibits no data reuse and is thus memory-bound at 5 TFLOP/s, the new kernel scales with problem size.

By disassembling the SASS assembly for both kernels, shown in Figure 19, we see that this change is directly attributable to the use of the TMA and WGMMA. While the naive kernel’s mainloop uses a series of serial scalar loads (LDG instructions) and scalar arithmetic operations (HMUL2 and FADD), the annotated kernel uses a combination of synchronization instructions (SYNCS, DEPBAR, ARRIVE), TMA operations (UTMALDG), and WGMMA invocations (HGMMA) to operate at full tile granularity.

```

1  — naive kernel (one thread per element)
2  /*0310*/ LDG.E.U16 R20, desc[UR6][R10.64] ;           // A[i*K + k]
3  /*0370*/ LDG.E.U16 R19, desc[UR6][R18.64] ;           // B[k*N + j]
4  /*0430*/ HMUL2 R19, R20.H0_H0, R19.H0_H0 ;           // scalar multiply
5  /*0460*/ FADD R21, R21, R4 ;                           // scalar accumulate
6
7  — new kernel: tiled TMA loads on thread 0
8  /*0410*/ SYNC.S.ARRIVE.TRANS64 RZ, [UR7+0xc000], R0 ; // TMA load barrier
9  /*0460*/ UTMALDG.2D [UR24], [UR36], desc[UR16] ;      // A tile load
10 /*0530*/ UTMALDG.2D [UR28], [UR34], desc[UR16] ;      // B tile load
11
12 — new kernels: batched WGMMMA on all threads
13 /*2dd0*/ SYNC.S.PHASECHK.TRANS64.TRYWAIT P0, [R3+UR4+0xc000], R0 ; // bar wait
14 /*2ee0*/ WARPGROUP.ARRIVE ;
15 /*2f30*/ HGMMMA.64x128x16.F32 R24, gdesc[UR16].tnspB, R24 ; // batch of WGMMMA
16 /*2f80*/ HGMMMA.64x128x16.F32 R24, gdesc[UR16].tnspB, R24 ;
17 /*2fd0*/ HGMMMA.64x128x16.F32 R24, gdesc[UR16].tnspB, R24 ;
18 /*3050*/ HGMMMA.64x128x16.F32 R24, gdesc[UR16].tnspB, R24, gsb0 ;
19 /*3060*/ WARPGROUP.DEPBAR.LE gsb0, 0x0 ;

```

Figure 19: The result of adding `WGMMMA_OUTER` and `WGMMMA_INNER` annotations to a naive loopnest is that BuildIt generates SASS that operates on tile-based granularity using native TMA and WGMMMA operations.

5.4.2 Better CTA Tiling

The first optimization beyond adding kernel annotations is to increase GPU occupancy, as the band-based tiling of CTAs only achieves an occupancy of 16 out of 132 SMs. Hence, we consider tiling the CTAs in both the M and the N dimensions, which is easy to do in BuildIt because of the affine loop-bound analyzer described in Section 4.4. We choose to thus have each CTA handle a 256×128 tile of the output matrix instead of a band of width 256 spanning the full N dimension, achieving full occupancy at problem sizes like $4096 \times 4096 \times 4096$. The code is displayed in Figure 20.

```

1  for (dyn_var<int> cta = 0; cta < (M/256)*(N/128); cta = cta + 1) {
2      ...
3      dyn_var<int> i_n = cta/(N/128);
4      dyn_var<int> j_n = cta%(N/128);
5      for (dyn_var<int> i = i_n*256; i < i_n*256 + 256; i = i + 1) {
6          for (dyn_var<int> j = j_n*128 + tid; j < j_n*128 + 128; j = j + 128)

```

Figure 20: Using modulo and division arithmetic enabled by the affine analyzer, we change the loop bounds so that each CTA handles a 256×128 output tile.

By just changing the dimensions of the loopnest, performance rises from 22.4 to 341.8 TFLOP/s at $4096 \times 4096 \times 4096$, reaching 45% of cuBLAS performance. Without needing to reason about any launch bounds, shared memory sizes, or code on the GPU, BuildIt enables the user to quickly and easily prototype different loopnest and tiling structures, enabling large performance improvements.

5.4.3 Warp Specialization and Tile Size Changes

Introducing warp specialization to the generated WGMMMA + TMA kernel through BuildIt is also a one-line change, which the user performs by raising the thread count from 128. In this section, we explore a kernel with 384 threads, or three warpgroups, which BuildIt leverages by invoking the warp-specialized `gemm_tile_ws` driver (from Section 4.6), with warpgroup 0 as a dedicated TMA producer and warpgroups 1/2 as WGMMMA consumers. Because warp-specialization enables register rebalancing between threads, the pass also increases the WGMMMA size from 64×128 to 64×256 and deepens the pipeline, as we can support greater register counts. Simultaneously, because the WGMMMA size has increased from 64×128 to 64×256 , we change the overall per-CTA tile size from 256×128 to 128×256 , as tiling 4 WGMMMA atoms in the M direction like before would lead to register and shared memory pressure.

This combination of changes directly improves the kernel to 422.9 TFLOP/s, reaching 56% of cuBLAS performance. Inspecting the assembly in Figure 21 once more, we see that the register rebalancing directives we use in warp specialization directly translate to SASS, become `USETMAXREG` instructions. Similarly, the choice to use larger matrix multiplications in BuildIt directly correlates to the increase in WGMMMA atom size, as seen in the `HGMMMA.64x256x16` instruction. In this way, by providing support for warp specialization and larger WGMMMA sizes (including by tuning the overall per-CTA tile size), BuildIt provides functionality for optimizing hardware parameters without requiring users to reason about hardware resource utilization, like for the register file.

```
1  — register rebalancing
2  /*03e0*/  USETMAXREG.TRY_ALLOC.CTAP00L UP0, 0xe8 ;
3  /*1740*/  USETMAXREG.DEALLOC.CTAP00L 0x28 ;
4
5  — consumer: larger size GMMMA instructions
6  /*0b10*/  HGMMMA.64x256x16.F32 R24, gdesc[UR12], R24 ;
7  /*0bc0*/  HGMMMA.64x256x16.F32 R24, gdesc[UR12], R24 ;
8  /*0c30*/  HGMMMA.64x256x16.F32 R24, gdesc[UR12], R24 ;
9  /*0c90*/  HGMMMA.64x256x16.F32 R24, gdesc[UR12], R24, gsb0 ;
```

Figure 21: The SASS changes in the warp-specialized kernel directly reflect register rebalancing through `USETMAXREG` instructions and larger WGMMMA atom sizes in the `HGMMMA.64x256x16` instructions.

5.4.4 Storing Matrices in the Right Layout

The final code changes we make are simple layout changes, changing `B[k*N + j]` to `B[j*K + k]` and `C[i*N + j]` to `C[j*M + i]`. These indexing modifications change B to K-major and C to M-major, resulting in significant changes to strides, TMA descriptors, and underlying CUDA machinery.

Functionally, these changes make a few key modifications that unlock key performance benefits. As depicted in Figure 22, when the matrix changes from N-major to K-major, the

WGMMA memory descriptor loses the `tnspB` annotation. This annotation is particularly important because it indicates that the WGMMA actually consumes the B matrix in K-major order, as the inner loop of the WGMMA loops over the K dimension. From a performance perspective, this small change to pack in the K dimension increases performance by 24%, reaching 524.5 TFLOP/s.

```

1 MN-major B WGMMA:
2   HGMMMA.64x256x16.F32 R24, gdesc[UR12].tnspB, R24 ;
3
4 K-major B WGMMA:
5   HGMMMA.64x256x16.F32 R24, gdesc[UR12], R24 ;

```

Figure 22: When the B matrix is stored in K-major order, it loses the `tnspB` annotation, indicating it lives in the native layout expected by the WGMMA.

Similar to the change of layout in the B matrix, changing the layout of the C matrix to be column-major (or M-major) unlocks better hardware utilization. As seen in Figure 23, each thread processes elements in adjacent columns, but not in adjacent rows. Specifically, each group of four threads processes two rows of the output tile, moving 8 columns at a time, with each thread processing two column-adjacent elements. However, this means that in the 8-column block, at one time step, only elements in non-adjacent columns would be processed simultaneously, which the GPU would not be able to coalesce. On the other hand, as seen in Figure 23, the two rows processed by each four thread group are 8 rows apart, and per the PTX documentation, each cluster of 8 rows is processed simultaneously by 8 groups of the aforementioned threads [14]. As a result, the GPU can potentially coalesce multiple elements in different rows, but not elements in different columns, making storing the C matrix in column-major order more optimal. This small change in input BuildIt code results in a 26% percent performance improvement, reaching a total of 661.7 TFLOP/s.

```

1 row-major C: each thread processes adjacent elements in each row
2 /*0f10*/ STG.E desc[UR10][R2.64],      R24 ; // (r, c)
3 /*0f20*/ STG.E desc[UR10][R2.64+0x4],  R25 ; // (r, c+1)
4 /*0f30*/ STG.E desc[UR10][R4.64],      R26 ; // (r+8, c)
5 /*0f40*/ STG.E desc[UR10][R4.64+0x4],  R27 ; // (r+8, c+1)
6
7 column-major C: each thread never touches adjacent elements in each column.
8 /*0f00*/ STG.E desc[UR10][R218.64],    R24 ; // (r, c)
9 /*0f20*/ STG.E desc[UR10][R220.64],    R25 ; // (r, c+1)
10 /*0f40*/ STG.E desc[UR10][R218.64+0x20], R26 ; // (r+8, c)
11 /*0f60*/ STG.E desc[UR10][R220.64+0x20], R27 ; // (r+8, c+1)

```

Figure 23: The final epilogue stores for the output matrix touch adjacent elements in each row but not each column, meaning only the column-major C can easily coalesce.

5.4.5 End-to-End Performance

After these optimizations, we summarize the final performance improvements below.

Table 5: Performance results from different BuildIt kernel configurations relative to cuBLAS. The main numbers are reported in TFLOP/s, and below each metric, we provide both the multiple of improvement over the naive kernel and the percent of the cuBLAS kernel achieved. Note that we report only the endpoints of the optimization sequence.

TFLOP/s (median)	512³	1024³	2048³	4096³	8192³	16384³
naive (1 thread / element)	3.9 (16%)	5.0 (3.2%)	5.3 (1.0%)	5.0 (0.7%)	2.9 (0.4%)	2.2 (0.3%)
generated, 1-D CTA band, 1 warpgroup	1.9 (0.5×, 8%)	4.8 (1.0×, 3%)	10.5 (2.0×, 2%)	22.4 (4.5×, 3%)	47.1 (16×, 6%)	95.3 (43×, 13%)
generated, 2-D tiling, 1 warpgroup	6.2 (1.6×, 26%)	32.0 (6.4×, 21%)	144.2 (27×, 28%)	341.8 (68×, 45%)	277.2 (96×, 36%)	197.3 (90×, 26%)
generated, all optimizations	15.1 (3.9×, 63%)	92.8 (19×, 60%)	496.6 (94×, 96%)	661.7 (132×, 87%)	639.0 (220×, 82%)	368.7 (168×, 49%)
cuBLAS baseline	24.0 (6.2×)	154.6 (31×)	517.2 (98×)	761.4 (152×)	776.8 (268×)	752.2 (342×)

As seen in Table 5, the final BuildIt generated kernel reaches 87% of cuBLAS performance for $4096 \times 4096 \times 4096$ GEMM, even reaching 96% of cuBLAS performance for $2048 \times 2048 \times 2048$ GEMM. Across the sweep, our final generated kernel achieves speedups between 3.9x and 220x over the naive baseline and ranges between 49% and 96% of cuBLAS, demonstrating the ability of BuildIt to write performant GEMM primitives using only loopnests. However, the percentage of cuBLAS achieved varies significantly across problem size, which we elaborate on below.

5.4.5.1 Small Problem Sizes

For small problem sizes, the chief issue is that with one CTA operating per WGMMMA tile of the output matrix, we cannot achieve full occupancy of the GPU, as there are less than 132 SMs worth of tiles to operate on. In contrast, existing algorithms like Stream-K require the kernel to also split work along the K-dimension of the matrix, allowing more CTAs to operate in parallel. However, this algorithmic flow does not easily align with BuildIt’s existing CUDA flow, as to accumulate the results in the K-dimension across multiple CTAs, the program must either launch an additional accumulation kernel or synchronize across multiple CTAs through global memory, neither of which is currently supported in BuildIt. As a result, we see that for sizes with M, N, K being either 512 or 1024, performance of our algorithm lags behind cuBLAS slightly.

5.4.5.2 Large Problem Sizes

Similar to small problem sizes, large problem sizes like $M = N = K = 16384$ produce unique challenges as well. Specifically, the loss comes from the L2 cache utilization, as a naive 2D walk of the output matrix no longer guarantees L2 cache hits in both of the

source matrices. To address this, existing algorithms use L2-aware walks of the output matrix, increasing L2 utilization as much as possible [15, 22]. However, even with L2-aware walks, CTAs that are initially launched together can drift apart across execution time, resulting in poor cache reuse in later CTA iterations. Existing algorithms can thus also use ideas like persistent threads, where exactly one CTA is launched per SM, to temporally align waves of computation, better utilizing the cache [7, 18]. However, BuildIt does not natively support inter-CTA communication, and we currently do not have an L2-tuned tiling for the kernel, leading us to lag behind on large problem sizes.

5.4.5.3 Intermediate Problem Sizes

For matrix sizes between 2048 and 8192, neither occupancy nor L2 utilization dominates, and instead, writing a performant kernel requires a full understanding of a synchronization, tiling, and the operations available on the GPU. On these problem sizes, BuildIt achieves its best performance, ranging from **82-96% of cuBLAS** and **94-220x improvement over the naive baseline**. These kernels are performant relative to state-of-the-art, and we attribute the slight gap in performance to advanced GPU features like multi-CTA cooperation, persistent grids, and TMA store, which we do not yet support, and discuss in more depth in Chapter 6. Even without these features, we still almost reach cuBLAS performance with a tuneable, simple loopnest representation [15, 22].

5.5 Final Thoughts

This chapter demonstrates that BuildIt CuTe allows the user to easily write and prototype performant GPU code. The user can choose different tiling schemes, memory layouts, and warp specialization, which each have real performance tradeoffs and map all the way down to concrete changes in SASS. At the same time, all the complex low-level scheduling details are abstracted away, meaning that we can nearly match cuBLAS performance with short, simple loopnests, like the final kernel shown below in Figure 24.

```

1  builder::annotate(CUDA_KERNEL);
2  for (dyn_var<int> cta = 0; cta < (M/128)*(N/256); cta = cta + 1) {
3      for (dyn_var<int> tid = 0; tid < 384; tid = tid + 1) {
4
5          builder::annotate(WGMMMA_OUTER);
6          for (dyn_var<int> i = (cta/(N/256))*128;
7              i < (cta/(N/256))*128 + 128; i = i + 1) {
8              for (dyn_var<int> j = (cta%(N/256))*256 + tid;
9                  j < (cta%(N/256))*256 + 256; j = j + 256) {
10
11                  builder::annotate(WGMMMA_INNER);
12                  for (dyn_var<int> k = 0; k < K; k = k + 1)
13                      C[j * M + i] = C[j * M + i] + A[i * K + k] * B[j * K + k];

```

Figure 24: This kernel, despite short, exhibits a number of design decisions, from the memory layout to warp specialization and tiling, that enable it to reach near cuBLAS performance.

Chapter 6

Limitations and Future Work

We finish the discussion of our work by covering the limitations of the current `extract_cute` pass and future directions for research.

6.1 Generalizing the Pass

6.1.1 Advanced Hardware Features

While the current pass generates kernels using TMA load and WGMMA, there are a number of other advanced hardware features on Hopper-generation GPUs that we currently do not leverage, as discussed in Section 5.4.5.

One of these features is **multi-CTA cooperation**, where thread-block clusters, TMA multicast, and persistent grids can share operand fetches across CTA boundaries or better schedule kernel prologues/epilogues. Just like with warp specialization, the outstanding question is how the pass should infer CTAs available for cooperation, instead of requiring the user to write cooperative code out of box. One possible idea is to infer which CTAs live adjacent to each other in the output matrix, enabling use of thread-block clusters, but the correct programming abstraction for this remains to be determined [15, 22].

Another such feature is **TMA store**, which replaces the register to global memory write at the end of the GEMM. While TMA store would help abstract away a significant portion of the epilogue, it would also require the pass to stage the output matrix in shared memory, as the TMA only operates on shared memory. To do so, the pass would have to reason more about how much shared memory remains available per SM, which we categorize as a potential future advancement to the pass [15, 22].

6.1.2 Divisibility Constraints

At the moment, the pass requires problem sizes to divide cleanly by the CTA tile size, and CTA tile sizes to divide cleanly by the WGMMA size. While this is good as a current proof-of-concept, further use would require remainder tiles, either through predicated TMA loads that mask out-of-bounds memory addresses, or by manually writing remainder loops

to clean up extra elements. Further work is required to isolate the performance tradeoff between these methods, and how these methods should be represented in the kernel emitter and affine analysis.

6.2 Fused Kernels

Current research on performant GPU code rarely performs isolated matrix multiplications, instead using matrix multiplications in larger, complex kernels. In these kernels, we do not want to load and store directly from global memory, instead piping computation prologues and epilogues fused around the GEMM. The main work in this direction primarily focuses on attention kernels, where softmax computations are fused around multiple matrix multiplications [3, 4, 20]. The current pass implementation is designed so that the WGMMMA region can sit inside a larger generated kernel, but we do not leverage shared memory or registers to pass data between different parts of a fused kernel. We believe this is a significant direction for generalizing BuildIt to a wider variety of kernels.

6.3 DSL Integration

A final potential direction for further research is integrating this thesis into an existing DSL for computing Einstein summations (or einsums). Einsums are a high level abstraction for representing matrix operations without having to write loopnests, and integrating our CuTe pass into a corresponding DSL would be a strong proof-of-concept for the usability of our code.

Supporting an einsum DSL is especially interesting as a direction for further work as it fundamentally ties into the discussion of abstraction versus complexity from the Introduction. The key idea of BuildIt is to allow the user to write extensible DSLs at the desired level of complexity, only inferring hardware primitives that the user should not have to reason about for high performance. For some users, the low-level abstraction that we provide in this thesis is thus the right interface, as they may want to write extensible kernels that leverage advanced hardware features like kernel fusion. Simultaneously, many users may want to use BuildIt to generate performant GPU code, and may not be familiar with GPU abstractions like CTAs, threads, and WGMMMA sizes. As such, the low-level interface provided in this thesis still enforces a number of hardware invariants, like thread counts, CTA tile sizes, and index expression formats. An einsum DSL would be able to further abstract this hardware complexity from the user, providing generic matrix support like in the Tensor Comprehensions paper [27].

Figure 25a: An einsum in BuildIt’s Einsum-Lang.

```
1 el::index i, j, k;
2 el::tensor<half_t> a({M, K}, A), b({K, N}, B), c({M, N}, C);
3 el::current_device = el::GPU_PARALLEL;
4 c[i][j] = a[i][k] * b[k][j];
```

Figure 25b: A sample DSL emitted loopnest for the einsum, taken from Figure 24.

```
1 builder::annotate(CUDA_KERNEL);
2 for (dyn_var<int> cta = 0; cta < (M/128)*(N/256); cta = cta + 1) {
3   for (dyn_var<int> tid = 0; tid < 384; tid = tid + 1) {
4
5     builder::annotate(WGMMMA_OUTER);
6     for (dyn_var<int> i = (cta/(N/256))*128;
7         i < (cta/(N/256))*128 + 128; i = i + 1) {
8       for (dyn_var<int> j = (cta%(N/256))*256 + tid;
9           j < (cta%(N/256))*256 + 256; j = j + 256) {
10
11         builder::annotate(WGMMMA_INNER);
12         for (dyn_var<int> k = 0; k < K; k = k + 1)
13           C[j * M + i] = C[j * M + i] + A[i * K + k] * B[j * K + k];
```

Figure 25c: The functional CUDA kernel extracted by the BuildIt CuTe pass.

```
1 cuda_kernel_0<<<(M/128)*(N/256), 384, kSmemBytes>>>(tma_a, tma_b, C, M, N, K);
```

Figure 25: The proposed einsum DSL integration, where (a), (b), and (c) display the writing of a matrix multiplication at the einsum DSL, BuildIt, and CUDA levels, respectively.

As we show in Figure 25, the einsum DSL would allow the user to just write the computation as a function of indices and matrices, where even the loopnests and loop dimensions are derived by the DSL compiler. This direction could also allow us to further explore autotuning, as we could use einsums as a simple IR frontend and the BuildIt pass as the compiler backend, which could find optimal tile sizes and memory layouts for maximal performance. Specifically, one potential avenue for further research is to define custom cost functions for all the Hopper-specific parameters we expose in the BuildIt CuTe pass, which would allow users to run the einsum DSL through a variety of solvers to find configurations with near state-of-the-art performance, like those presented in Section 5.4.

Chapter 7

Conclusion

In this thesis, we propose a novel compiler extension that leverages BuildIt’s annotation-driven model to convert plain loopnests into performant, asynchronous kernels for Hopper-generation GPUs that coordinate across the TMA and WGMMMA hardware coprocessors.

To implement this extension, we introduce custom annotations that allow BuildIt to reason about GPU memory primitives, a custom staged CuTe helper that abstracts much of the hardware complexity away from the BuildIt compiler, and a custom affine analyzer that reasons about loop bounds and tile sizes. Using these compiler extensions, the `extract_cute` pass in BuildIt can take a fifteen-line loopnest specification and extract thread block tiling constraints, memory layouts, operand data types, batching requirements, and warpgroup allocations. These extracted values can then be used to emit performant GEMM kernels with 20x less user-written code than a hand-rolled CUDA kernel. Instead, BuildIt chooses to assign the majority of the kernel writing work within the framework itself.

Across nine configurations of data types, memory layouts, batching, CTA tiling, and warpgroup allocations, generated kernels are within epsilon to cuBLAS over a wide sweep of matrix sizes. Additionally, BuildIt allows the user to flexibly change many tiling and layout constraints natively within the loopnest, and when optimizing for performance, generated kernels reach up to 96% of the performance achieved by cuBLAS, compared to 1.0% without the new extraction pass.

As a whole, this thesis thus demonstrates that flexible, performant kernels for DSLs can be efficiently generated directly from loopnests in BuildIt, providing a strong baseline for future development of high-performance GPU programming models and languages.

Bibliography

- [1] Ajay Brahmakshatriya and Saman Amarasinghe. 2021. BuildIt: A Type-Based Multi-stage Programming Framework for Code Generation in C++. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2021. 39–51. <https://doi.org/10.1109/CGO51591.2021.9370333>
- [2] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Meghan Cowan, Haichen Shen, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: an automated end-to-end optimizing compiler for deep learning. In *Proceedings of the 13th USENIX Conference on Operating Systems Design and Implementation (OSDI'18)*, 2018. USENIX Association, Carlsbad, CA, USA, 579–594.
- [3] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [4] Tri Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *International Conference on Learning Representations (ICLR)*, 2024.
- [5] Yaoyao Ding, Cody Hao Yu, Bojian Zheng, Yizhi Liu, Yida Wang, and Gennady Pekhimenko. 2023. Hidet: Task-Mapping Programming Paradigm for Deep Learning Tensor Programs. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS 2023)*, 2023. Association for Computing Machinery, Vancouver, BC, Canada, 370–384. <https://doi.org/10.1145/3575693.3575702>
- [6] Siyuan Feng, Bohan Hou, Hongyi Jin, Wuwei Lin, Junru Shao, Ruihang Lai, Zihao Ye, Lianmin Zheng, Cody Hao Yu, Yong Yu, and Tianqi Chen. 2023. TensorIR: An Abstraction for Automatic Tensorized Program Optimization. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS 2023)*, 2023. Association for Computing Machinery, Vancouver, BC, Canada, 804–817. <https://doi.org/10.1145/3575693.3576933>
- [7] Kshitij Gupta, Jeff A. Stuart, and John D. Owens. 2012. A Study of Persistent Threads Style GPU Programming for GPGPU Workloads. In *2012 Innovative*

- Parallel Computing (InPar)*, 2012. IEEE, 1–14. <https://doi.org/10.1109/InPar.2012.6339596>
- [8] Bastian Hagedorn, Archibald Samuel Elliott, Henrik Barthels, Rastislav Bodik, and Vinod Grover. 2020. Fireiron: A Data-Movement-Aware Scheduling Language for GPUs. In *Proceedings of the ACM International Conference on Parallel Architectures and Compilation Techniques (PACT '20)*, 2020. Association for Computing Machinery, Virtual Event, GA, USA, 71–82. <https://doi.org/10.1145/3410463.3414632>
 - [9] Bastian Hagedorn, Bin Fan, Hanfeng Chen, Cris Cecka, Michael Garland, and Vinod Grover. 2023. Graphene: An IR for Optimized Tensor Computations on GPUs. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3 (ASPLOS 2023)*, 2023. Association for Computing Machinery, Vancouver, BC, Canada, 302–313. <https://doi.org/10.1145/3582016.3582018>
 - [10] Yuka Ikarashi, Kevin Qian, Samir Droubi, Alex Reinking, Gilbert Louis Bernstein, and Jonathan Ragan-Kelley. 2025. Exo 2: Growing a Scheduling Language. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '25)*, 2025. Association for Computing Machinery, Rotterdam, Netherlands, 426–444. <https://doi.org/10.1145/3669940.3707218>
 - [11] NVIDIA. 2022. NVIDIA H100 Tensor Core GPU Architecture. Retrieved August 11, 2026 from <https://resources.nvidia.com/en-us-hopper-architecture/nvidia-h100-tensor-c>
 - [12] NVIDIA. 2026. CUTLASS (Version 4.7.0). Retrieved August 11, 2026 from <https://github.com/NVIDIA/cutlass>
 - [13] NVIDIA. 2026. CUDA C++ Programming Guide. Retrieved August 11, 2026 from <https://docs.nvidia.com/cuda/cuda-programming-guide/>
 - [14] NVIDIA. 2026. Parallel Thread Execution ISA, Version 9.3. Retrieved August 11, 2026 from <https://docs.nvidia.com/cuda/parallel-thread-execution/>
 - [15] NVIDIA. 2026. NVIDIA CUTLASS Documentation. Retrieved August 11, 2026 from <https://docs.nvidia.com/cutlass/latest/overview.html>
 - [16] NVIDIA. 2026. cuTile Python Documentation. Retrieved August 11, 2026 from <https://docs.nvidia.com/cuda/cutile-python/>
 - [17] NVIDIA. 2026. CuTe DSL Documentation. Retrieved August 11, 2026 from https://docs.nvidia.com/cutlass/latest/media/docs/pythonDSL/cute_dsl.html
 - [18] Muhammad Osama, Duane Merrill, Cris Cecka, Michael Garland, and John D. Owens. 2023. Stream-K: Work-Centric Parallel Decomposition for Dense Matrix-

- Matrix Multiplication on the GPU. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP '23)*, 2023. Association for Computing Machinery, New York, NY, USA, 429–431. <https://doi.org/10.1145/3572848.3577479>
- [19] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. 2013. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '13)*, 2013. Association for Computing Machinery, Seattle, Washington, USA, 519–530. <https://doi.org/10.1145/2491956.2462176>
 - [20] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision. In *Advances in Neural Information Processing Systems*, 2024. Curran Associates, Inc., 68658–68685. <https://doi.org/10.52202/079017-2193>
 - [21] Pranjal Shankhdhar. 2024. fast.cu: Fastest Kernels Written from Scratch. Retrieved August 11, 2026 from <https://github.com/pranjalssh/fast.cu>
 - [22] Pranjal Shankhdhar. 2024. Outperforming cuBLAS on H100: a Worklog. Retrieved August 11, 2026 from <https://cudaforfun.substack.com/p/outperforming-cublas-on-h100-a-worklog>
 - [23] Gina Sohn, Genghan Zhang, Konstantin Hossfeld, Jungwoo Kim, Nathan Sobotka, Nathan Zhang, Olivia Hsu, and Kunle Olukotun. 2026. Streaming Tensor Programs: A Streaming Abstraction for Dynamic Parallelism. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26)*, 2026. Association for Computing Machinery, USA, 1912–1932. <https://doi.org/10.1145/3779212.3790229>
 - [24] Benjamin Spector, Simran Arora, Aaryan Singhal, Arjun Parthasarathy, Dan Fu, and Christopher Re. 2025. ThunderKittens: Simple, Fast, and Adorable Kernels. In *International Conference on Learning Representations*, 2025. 2097–2130. Retrieved from https://proceedings.iclr.cc/paper_files/paper/2025/file/05dc08730e32441edff52b0fa6caab5f-Paper-Conference.pdf
 - [25] Arvind K. Sujeeth, Kevin J. Brown, Hyoukjoong Lee, Tiark Rompf, Hassan Chafi, Martin Odersky, and Kunle Olukotun. 2014. Delite: A Compiler Architecture for Performance-Oriented Embedded Domain-Specific Languages. *ACM Trans. Embed. Comput. Syst.* 13, 4s (April 2014). <https://doi.org/10.1145/2584665>
 - [26] Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming*

- Languages (MAPL 2019)*, 2019. Association for Computing Machinery, Phoenix, AZ, USA, 10–19. <https://doi.org/10.1145/3315508.3329973>
- [27] Nicolas Vasilache, Oleksandr Zinenko, Theodoros Theodoridis, Priya Goyal, Zachary DeVito, William S. Moses, Sven Verdoolaege, Andrew Adams, and Albert Cohen. 2018. Tensor Comprehensions: Framework-Agnostic High-Performance Machine Learning Abstractions. *CoRR* (2018). Retrieved from <http://arxiv.org/abs/1802.04730>
 - [28] Lei Wang, Yu Cheng, Yining Shi, Zhiwen Mo, Zhengju Tang, Wenhao Xie, Tong Wu, Lingxiao Ma, Yuqing Xia, Jilong Xue, Fan Yang, and Zhi Yang. 2026. TileLang: Bridge Programmability and Performance in Modern Neural Kernels. In *International Conference on Learning Representations*, 2026. 73099–73134. Retrieved from https://proceedings.iclr.cc/paper_files/paper/2026/file/76fb92288bf90360c527efb0d1c2aba6-Paper-Conference.pdf
 - [29] Jian Weng, Animesh Jain, Jie Wang, Leyuan Wang, Yida Wang, and Tony Nowatzki. 2021. UNIT: unifying tensorized instruction compilation. In *Proceedings of the 2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO '21)*, 2021. IEEE Press, Virtual Event, Republic of Korea, 77–89. <https://doi.org/10.1109/CGO51591.2021.9370330>
 - [30] Mengdi Wu, Xinhao Cheng, Shengyu Liu, Chunan Shi, Jianan Ji, Man Kit Ao, Praveen Velliengiri, Xupeng Miao, Oded Padon, and Zhihao Jia. 2025. Mirage: A Multi-Level Superoptimizer for Tensor Programs. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, July 2025. USENIX Association, Boston, MA, 21–38. Retrieved from <https://www.usenix.org/conference/osdi25/presentation/wu-mengdi>
 - [31] Jiarong Xing, Leyuan Wang, Shang Zhang, Jack Chen, Ang Chen, and Yibo Zhu. 2022. Bolt: Bridging the Gap between Auto-tuners and Hardware-native Performance. In *Proceedings of Machine Learning and Systems*, 2022. 204–216. Retrieved from https://proceedings.mlsys.org/paper_files/paper/2022/file/1f8053a67ec8e0b57455713cefdd8218-Paper.pdf
 - [32] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. In *Proceedings of Machine Learning and Systems*, 2025. MLSys, . Retrieved from https://proceedings.mlsys.org/paper_files/paper/2025/file/dbf02b21d77409a2db30e56866a8ab3a-Paper-Conference.pdf
 - [33] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. 2020. Ansor: Generating High-Performance Tensor Programs for Deep

- Learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, November 2020. USENIX Association, 863–879. Retrieved from <https://www.usenix.org/conference/osdi20/presentation/zheng>
- [34] Size Zheng, Renze Chen, Anjiang Wei, Yicheng Jin, Qin Han, Liqiang Lu, Bingyang Wu, Xiuhong Li, Shengen Yan, and Yun Liang. 2022. AMOS: enabling automatic mapping for tensor computations on spatial accelerators with hardware abstraction. In *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA '22)*, 2022. Association for Computing Machinery, New York, New York, 874–887. <https://doi.org/10.1145/3470496.3527440>
- [35] Hongyu Zhu, Ruofan Wu, Yijia Diao, Shanbin Ke, Haoyu Li, Chen Zhang, Jilong Xue, Lingxiao Ma, Yuqing Xia, Wei Cui, Fan Yang, Mao Yang, Lidong Zhou, Asaf Cidon, and Gennady Pekhimenko. 2022. ROLLER: Fast and Efficient Tensor Compilation for Deep Learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, July 2022. USENIX Association, Carlsbad, CA, 233–248. Retrieved from <https://www.usenix.org/conference/osdi22/presentation/zhu>